



Hochschule Hannover

Fakultät II – Maschinenbau und Bioverfahrenstechnik

Labor für Produktentwicklung und Digitale Fabrik (LFP)

Bachelorarbeit

Potenziale und Herausforderungen Digitaler Zwillinge in der Fabrikplanung mit NVIDIA Omniverse

Vorgelegt von:	Tobias Küster
Matrikelnummer:	1690767
E-Mail:	tobias.kuester@stud.hs-hannover.de
Studiengang:	Ingenieurinformatik - Maschinenbau (IIM, B.Eng.)
Fachsemester:	10
Semester:	SoSe 2026
Erstprüfer:	Prof. Dr.-Ing. P. Diersen
Zweitprüfer:	Prof. Dr.-Ing. A. Vendl
Betreuer LFP:	A. Ernst
Ausgabe:	19.06.2026
Abgabe:	10.08.2026

Kurzfassung

Die fortschreitende Digitalisierung industrieller Produktion erfordert dynamische Planungsansätze, in denen Digitale Zwillinge als Bindeglied zwischen physischer Anlage und virtuellem Modell fungieren. Die vorliegende Arbeit untersucht die Potenziale und Herausforderungen, die sich aus der Nutzung von NVIDIA Omniverse sowie der darauf aufbauenden Simulationsplattform Isaac Sim (im Folgenden: Isaac Sim) für die Fabrikplanung ergeben.

Im Labor für Produktentwicklung und Digitale Fabrik der Hochschule Hannover werden auf Basis dieser Plattformen Digitale Zwillinge zweier realer Anlagen entwickelt: ein Rundtaktisch („Maze Runner“) sowie der Gelenkarmroboter „Dobot Magician“. Die Modellierung erfolgt unter Nutzung des offenen Datenformats OpenUSD sowie der Middleware-Architekturen OPC UA, MQTT und ROS 2 in Rahmen einer Python basierten Extension für NVIDIA Omniverse.

Ergänzend wird eine methodische Anleitung zur Reproduzierbarkeit der Modellerstellung entwickelt, sodass Studierende und industrielle Anwender den Prozess eigenständig nachvollziehen können. Die Arbeit schließt mit einer kritischen Diskussion infrastruktureller, ökonomischer und qualifikatorischer Anforderungen sowie der Formulierung strategischer Handlungsempfehlungen für Industrieunternehmen und Hochschulen.

Schlagnworte: Digitaler Zwilling, NVIDIA Omniverse, Isaac Sim, OpenUSD, Fabrikplanung, virtuelle Inbetriebnahme, OPC UA, ROS 2.

Abstract

The ongoing digitalisation of industrial production calls for dynamic planning approaches in which digital twins serve as the interface between physical assets and their virtual representation. This thesis investigates the potentials and challenges that arise from applying NVIDIA Omniverse and its simulation platform Isaac Sim (hereafter referred to as Isaac Sim) to factory planning. Within the Laboratory of Product Development and Digital Factory at Hannover University of Applied Sciences and Arts, digital twins of two real installations are developed: a rotary indexing table (“Maze Runner”), provided by Yübo Wang (Hochschule RheinMain / Mercedes-Benz), who also contributed to shaping the contents of this project, and the articulated robot arm “Dobot Magician”. Modelling is based on the open data format OpenUSD and on the middleware architectures OPC UA, MQTT and ROS 2, implemented within a Python-based extension for NVIDIA Omniverse. A methodological guideline ensures reproducibility, enabling students and industrial users to recreate the modelling workflow autonomously. The thesis concludes with a critical discussion of infrastructural, economic, and qualificational requirements as well as strategic recommendations for industry and higher education.

Keywords: digital twin, NVIDIA Omniverse, Isaac Sim, OpenUSD, factory planning, virtual commissioning, OPC UA, ROS 2.

Aufgabenstellung

Titel: Potenziale und Herausforderungen Digitaler Zwillinge in der Fabrikplanung mit NVIDIA Omniverse

Bearbeiter: Tobias Küster

Erstprüfer: Prof. Dr.-Ing. P. Diersen, Labor für Produktentwicklung und Digitale Fabrik

Zweitprüfer: Prof. Dr.-Ing. A. Vendl

Betreuer LFP: A. Ernst

Ausgabe: 19.06.2026

Industrie 4.0 und cyber-physische Produktionssysteme stellen die Fabrikplanung vor neue Anforderungen. Hohe Variantenvielfalt, kurze Produktzyklen und steigende Wandlungsfähigkeit verlangen dynamische statt statischer Planungsansätze. Digitale Zwillinge fungieren dabei als zentrale Technologie durch ihr kontinuierliches Abbild realer Systeme. Mit NVIDIA Omniverse und Isaac Sim entstehen Plattformen, die durch offene Standards, physikalische Genauigkeit und fotorealistisches Rendering neue Maßstäbe in der Simulation setzen. Daher sollen im Labor für Produktentwicklung und Digitale Fabrik (LFP) der Hochschule Hannover von zwei realen Anlagen Digitale Zwillinge erstellt werden.

Teilziele:

- Recherche notwendiger theoretischer Grundlagen (Digitalisierungsstufen, Isaac Sim, Middleware OPC UA / MQTT / ROS 2).
- Entwicklung Digitaler Zwillinge:
 1. Rundtakttisch „Maze Runner“ (Fertigung eines Produktes)
 2. Gelenkarmroboter „Dobot Magician“ (Pick and Place Szenario)
- Inbetriebnahme und Optimierung des Maze Runner und Dobot Magician
- Reproduzierbarkeitsdokumentation (PowerPoint, Video, VR/AR).
- Ableitung von Potenzialen, Herausforderungen und Handlungsempfehlungen.
- Vollständige Dokumentation (max. 50 Seiten) und digitale Übergabe an das LFP; Open-Access-Veröffentlichung über die Bibliothek der HsH angedacht.

Selbständigkeitserklärung

Hiermit versichere ich, Tobias Küster, dass ich die vorliegende Bachelorarbeit mit dem Titel „Potenziale und Herausforderungen Digitaler Zwillinge in der Fabrikplanung mit NVIDIA Omniverse“ selbständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Die aus fremden Quellen – einschließlich elektronischer Quellen – direkt oder indirekt übernommenen Gedanken sind ausnahmslos unter genauer Quellenangabe kenntlich gemacht. Der Einsatz von KI-basierten Werkzeugen wurde gemäß Anhang A dokumentiert. Die Arbeit ist in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht worden.

Ort, Datum

Unterschrift (Tobias Küster)

Inhaltsverzeichnis

Kurzfassung	II
Abstract	II
Aufgabenstellung	III
Selbständigkeitserklärung	IV
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VII
Abkürzungsverzeichnis	VIII
1. Einleitung	1
1.1. Ausgangssituation und Motivation	1
1.2. Problemstellung	1
1.3. Zielsetzung und Forschungsfragen	1
1.4. Vorstellung des Labors für Produktentwicklung und Digitale Fabrik	2
1.5. Aufbau der Arbeit	2
2. Theoretische Grundlagen	2
2.1. Fabrikplanung und Digitale Fabrik	2
2.2. Digitalisierungsstufen und Klassifikation Digitaler Zwillinge	3
2.2.1. Klassifikation nach Kritzinger et al.	3
2.2.2. Definition nach Stark und Damerau	3
2.2.3. Normativer Rahmen: ISO 23247	3
2.2.4. Referenzarchitekturmodell RAMI 4.0	4
2.2.5. Verwaltungsschale (Asset Administration Shell)	4
2.3. NVIDIA Omniverse-Ökosystem	5
2.4. Datenformate: OpenUSD und URDF	6
2.5. Middleware-Architekturen für die industrielle Kommunikation	7
2.5.1. OPC UA (DIN EN IEC 62541)	7
2.5.2. MQTT v5.0	7
2.5.3. ROS 2 und DDS	7
2.6. Virtuelle Inbetriebnahme nach VDI/VDE 3693	7
3. Methodik und Vorgehensweise	8
3.1. Vorgehensweise	8
3.2. Anforderungsanalyse	8
3.3. Toolkette	9
3.4. Zeit- und Arbeitsplan	9
4. Entwicklung der Digitalen Zwillinge	10
4.1. Anlage 1: Rundtakttisch „Maze Runner“	10
4.1.1. Beschreibung der Anlage	10

4.1.2.	Geometrische Modellierung und USD-Konvertierung	10
4.1.3.	Kinematik und Physik in Isaac Sim	11
4.1.4.	Architektur der Omniverse-Extension	11
4.1.5.	SPS-Anbindung und Kommunikationsarchitektur	12
4.1.6.	Inbetriebnahme und Optimierung	14
4.1.7.	Diskussion der Ergebnisse	14
4.2.	Anlage 2: Desktoproboter „Dobot Magician“ – Pick-and-Place-Szenario	15
4.2.1.	Beschreibung der Anlage	15
4.2.2.	Problemstellung und Konzept für den Digitalen Zwilling	15
4.2.3.	URDF-Modellierung und Import in Isaac Sim	16
4.2.4.	Systemarchitektur und ROS 2-Anbindung	17
4.2.5.	Inbetriebnahme	17
4.2.6.	Diskussion der Ergebnisse	18
5.	Dokumentation der Entwicklungspipeline	18
5.1.	Omniverse-Extension und ROS 2-Anbindung als Entwicklungsergebnisse	18
5.2.	Reproduzierbarkeitsdokumentation	19
5.2.1.	Methodische Schritt-für-Schritt-Anleitung – Maze Runner	19
5.2.2.	Reproduzierbarkeitsdokumentation Dobot Magician	19
5.2.3.	Medienformate	19
5.2.4.	Übergabe an das LFP	20
5.3.	Gegenüberstellung und Gesamtbewertung	20
6.	Potenziale, Herausforderungen und Handlungsempfehlungen	20
6.1.	Potenziale im Labor für Produktentwicklung und Digitale Fabrik	20
6.1.1.	Potenziale Digitaler Fabriken	20
6.1.2.	Potenziale von Gelenkarmrobotern	21
6.2.	Herausforderungen im Laborbetrieb	21
6.2.1.	Infrastruktur	21
6.2.2.	Qualifikation	21
6.3.	Handlungsempfehlungen für das LFP	21
7.	Zusammenfassung und Ausblick	22
7.1.	Zusammenfassung	22
7.2.	Ausblick	22
	Literatur	23
	A. Verwendete KI-Software	26
	B. Reproduktions-Anleitung (Auszug)	26
	C. Auszug aus der URDF-Beschreibung des Dobot	26
	D. Küster Omniverse Tutorial – Reproduktionsanleitung	27
	E. Schnittstellenmatrix Maze Runner (Auszug)	80

Abbildungsverzeichnis

1.	Digital Twin Framework nach ISO 23247: Struktur aus User Entity, Core Entity, Data Collection and Device Control Entity sowie Cross-System Entity mit den jeweiligen funktionalen Elementen (FE) (Quelle: ISO 23247-2 [11]).	4
2.	RAMI 4.0 – Referenzarchitekturmodell Industrie 4.0 als dreidimensionales Schichtenmodell mit den Achsen „Life Cycle & Value Stream“, „Hierarchy Levels“ und „Layers“ (Quelle: DIN SPEC 91345 [3]).	5
3.	OpenUSD-Modell des Maze Runners in NVIDIA Isaac Sim (Viewport, Stage-Baum und Property-Panel).	11
4.	Verschachtelte Gelenke am Schwenkarm-Deckel: <i>RevoluteJoint rotatory</i> (links) und <i>PrismaticJoint translatory</i> (rechts) mit Kollisionskonturen (grün).	12
5.	Systemarchitektur des Maze-Runner-Digitalen-Zwillings: SPS-Netzwerk (links), Omniverse-Extension (Mitte) und DigitalTwinService mit VPN-Gateway (rechts). .	13
6.	IST-Zustand der Maze-Runner-Anlage im Hochschullabor (Aufnahme von oben). .	14

Tabellenverzeichnis

1.	Anforderungsliste für die Anforderungsanalyse	8
2.	Zeit- und Arbeitsplan der Bachelorarbeit	9
3.	Übersicht verwendeter KI-Software	26
4.	Sieben-Schritte-Modell zur Reproduktion eines Digitalen Zwillings	80
5.	Auszug aus der OPC-UA-Schnittstellenmatrix des Maze Runner	80

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AAS	Asset Administration Shell (Verwaltungsschale)
API	Application Programming Interface
CAD	Computer-Aided Design
CPPS	Cyber-Physical Production System
DDS	Data Distribution Service
DT	Digital Twin (Digitaler Zwilling)
GPU	Graphics Processing Unit
HsH	Hochschule Hannover
IDTA	Industrial Digital Twin Association
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
KI	Künstliche Intelligenz
LFP	Labor für Produktentwicklung und Digitale Fabrik
LLM	Large Language Model
MQTT	Message Queuing Telemetry Transport
OPC UA	Open Platform Communications Unified Architecture
OpenUSD	Universal Scene Description
QoS	Quality of Service
RAMI	Reference Architecture Model Industrie 4.0
REST	Representational State Transfer
ROS 2	Robot Operating System 2
SCARA	Selective Compliance Assembly Robot Arm
SPS	Speicherprogrammierbare Steuerung
TSN	Time-Sensitive Networking
URDF	Unified Robot Description Format
VDI	Verein Deutscher Ingenieure
VIBN	Virtuelle Inbetriebnahme
VPN	Virtual Private Network
VR	Virtual Reality
WSL	Windows Subsystem for Linux

1. Einleitung

1.1. Ausgangssituation und Motivation

Die produzierende Industrie befindet sich in einem tiefgreifenden strukturellen Wandel. Steigende Variantenvielfalt, verkürzte Produktlebenszyklen und die Forderung nach individualisierten Erzeugnissen verlangen von Fabriken eine kontinuierlich wachsende Wandlungsfähigkeit. Klassische, statische Planungsansätze stoßen dabei an ihre Grenzen, da sie weder mit der erforderlichen Geschwindigkeit auf Marktveränderungen reagieren noch die zunehmende Komplexität verteilter Produktionssysteme adäquat abbilden können [1]. Monostori beschreibt die hieraus resultierenden cyber-physischen Produktionssysteme (CPPS) als enge „Kopplung und fundamentale Verschränkung physischer Operationen mit softwaretechnischen und kommunikationsorientierten Netzwerken“ [2]. Das strukturelle Pendant zu dieser Entwicklung bildet das Referenzarchitekturmodell RAMI 4.0, das in DIN SPEC 91345 als dreidimensionales Schichtenmodell verankert ist und Kommunikationsstrukturen sowie eine gemeinsame Begriffsbasis für Industrie 4.0 definiert [3].

1.2. Problemstellung

Die in CPPS anfallenden Datenmengen und die wachsende Heterogenität der eingesetzten Engineering-Werkzeuge führen zu Medienbrüchen entlang des gesamten Lebenszyklus einer Fabrik. Geometriedaten, Steuerungsinformationen und Verhaltensmodelle liegen typischerweise in proprietären Formaten verteilt vor, was den Aufbau eines konsistenten, in Echtzeit aktualisierten digitalen Abbildes erschwert. Bestehende Simulationsumgebungen bieten zwar leistungsfähige Einzelfunktionen, scheitern jedoch häufig an der Skalierbarkeit im Dauerbetrieb sowie an der Integration heterogener Datenquellen [4]. Mit NVIDIA Omniverse und Isaac Sim sind Plattformen verfügbar, die durch das offene Datenformat OpenUSD, GPU-beschleunigte Physik und fotorealistisches Rendering einen alternativen, integrierten Ansatz versprechen [5], [6]. Inwieweit diese Plattformen die Anforderungen der Fabrikplanung erfüllen und welche Herausforderungen sich daraus ergeben, ist bislang nur unzureichend systematisch untersucht.

1.3. Zielsetzung und Forschungsfragen

Ziel der vorliegenden Arbeit ist es, Potenziale und Herausforderungen der Integration Digitaler Zwillinge in die Fabrikplanung am Beispiel von NVIDIA Omniverse und Isaac Sim systematisch zu untersuchen. Hierzu werden im Labor für Produktentwicklung und Digitale Fabrik (LFP) der Hochschule Hannover Digitale Zwillinge zweier realer Anlagen entwickelt – ein Rundtaktstisch („Maze Runner“) und der Desktop-Roboter „Dobot“. Auf dieser Basis werden folgende Forschungsfragen bearbeitet:

1. Welche normativen und konzeptionellen Grundlagen sind für die Erstellung Digitaler Zwillinge in der Fabrikplanung maßgebend?

2. Wie lassen sich reale Produktionsanlagen unter Nutzung von OpenUSD, Isaac Sim und industrieller Middleware (OPC UA, MQTT, ROS 2) als Digitale Zwillinge abbilden?
3. Wie kann die Reproduzierbarkeit eines solchen Erstellungsprozesses für nachfolgende Anwender gewährleistet werden?
4. Welche Handlungsempfehlungen lassen sich für Industrieunternehmen und Hochschulen ableiten?

1.4. Vorstellung des Labors für Produktentwicklung und Digitale Fabrik

Das Labor für Produktentwicklung und Digitale Fabrik (LFP) der Fakultät II der Hochschule Hannover bündelt Forschung und Lehre in den Themenfeldern methodische Produktentwicklung, virtuelle Fabrikplanung und digitale Prozessketten. Die vorhandene Infrastruktur umfasst CAD- und Simulationsarbeitsplätze, Workstations mit RTX-GPU-Beschleunigung für den Einsatz von NVIDIA Omniverse, eine VR-/AR-Demoumgebung sowie reale Demonstrationsanlagen zur Validierung digitaler Modelle.

1.5. Aufbau der Arbeit

Die Arbeit gliedert sich in sieben Kapitel. Nach der Einleitung legt Kapitel 2 die theoretischen Grundlagen und den normativen Rahmen dar. Kapitel 3 beschreibt die Methodik nach VDI 2222 sowie den Zeit- und Arbeitsplan. In Kapitel 4 erfolgt die Entwicklung der Digitalen Zwillinge der beiden Anlagen, Kapitel ?? behandelt Inbetriebnahme, Reproduzierbarkeit und Diskussion der Ergebnisse. Kapitel 6 fasst Potenziale, Herausforderungen und Handlungsempfehlungen zusammen, bevor Kapitel 7 eine abschließende Bewertung und einen Ausblick gibt.

2. Theoretische Grundlagen

2.1. Fabrikplanung und Digitale Fabrik

Die Richtlinie VDI 5200 Blatt 1 gliedert die Fabrikplanung in sieben sequenzielle Phasen und liefert damit das methodische Grundgerüst, das in dieser Arbeit durch digitale Methoden erweitert wird [7]. VDI 4499 Blatt 1 definiert die Digitale Fabrik als vernetztes System aus digitalen Modellen, Methoden und Werkzeugen zur ganzheitlichen Planung und Steuerung von Fabrikprozessen [8]. Cyber-physische Produktionssysteme (CPPS) ergänzen diesen Rahmen um die bidirektionale Echtzeit-Kopplung zwischen physischer Anlage und digitalem Abbild und bilden damit die technische Voraussetzung für Digitale Zwillinge im engeren Sinne [2]. Der industrielle Bedarf an solchen Lösungen ist branchenübergreifend belegt – exemplarisch zeigt die Automobilindustrie, dass durchgängige digitale Fabrikmodelle Planungszeiten signifikant verkürzen und Anlauf Risiken reduzieren [4].

2.2. Digitalisierungsstufen und Klassifikation Digitaler Zwillinge

2.2.1. Klassifikation nach Kritzinger et al.

Die in der Literatur am häufigsten herangezogene Differenzierung der Digitalisierungsstufen stammt von Kritzinger et al. und unterscheidet anhand des Datenflusses zwischen physischem und digitalem Objekt drei Reifegrade [9]:

- **Digitales Modell:** Keine automatisierte Datenkopplung; jegliche Aktualisierung zwischen physischem und digitalem Objekt erfolgt manuell.
- **Digitaler Schatten:** Ein automatisierter, unidirektionaler Datenfluss vom physischen zum digitalen Objekt; das digitale Objekt spiegelt den Zustand des physischen wider, wirkt aber nicht auf dieses zurück.
- **Digitaler Zwilling:** Ein „vollständig integrierter, in beide Richtungen automatisierter Datenfluss zwischen dem physischen und dem digitalen Objekt“ [9]. Veränderungen am physischen Objekt führen unmittelbar zu Anpassungen des digitalen Abbildes – und umgekehrt.

Diese Klassifikation dient in der vorliegenden Arbeit als Maßstab zur Einordnung der entwickelten Lösungen.

2.2.2. Definition nach Stark und Damerau

Ergänzend liefert die CIRP Encyclopedia of Production Engineering durch Stark und Damerau eine lexikalisch verbindliche Definition, nach der ein Digitaler Zwilling „ein digitales Abbild eines aktiven, einzigartigen Produkts ... oder Produktionssystems ist, das durch ausgewählte Eigenschaften, Bedingungen und Verhaltensweisen mittels Modellen, Informationen und Daten charakterisiert wird“ [10]. Damit wird der Aspekt der Einzigartigkeit – jedes physische Objekt besitzt genau einen Zwilling – explizit hervorgehoben.

2.2.3. Normativer Rahmen: ISO 23247

Die Normenreihe ISO 23247 stellt erstmals einen international abgestimmten Rahmen für Digitale Zwillinge in der Fertigung bereit. Teil 1 definiert den Digitalen Zwilling als synchronisierte Repräsentation eines „observable manufacturing element“ und legt allgemeine Grundsätze, Anwendungsfälle und Anforderungen fest [11]. Die weiteren Teile spezifizieren die Referenzarchitektur (Teil 2), die Klassifikation fertigungsrelevanter Elemente (Teil 3) sowie den Informationsaustausch (Teil 4). Abbildung 1 zeigt das zugrunde liegende *Digital Twin Framework* mit seinen Entitäten und funktionalen Elementen. Für die vorliegende Arbeit bildet ISO 23247 die maßgebende normative Grundlage, an der die entwickelten Lösungen gemessen werden.

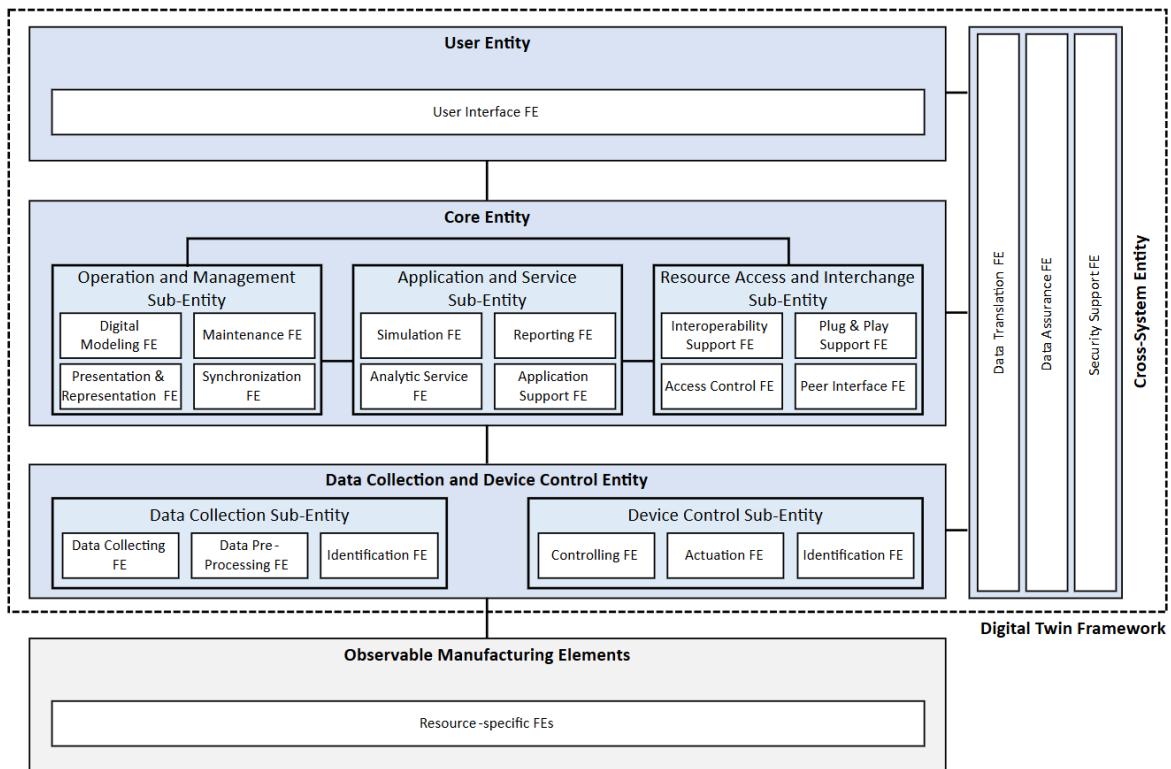


Abbildung 1: Digital Twin Framework nach ISO 23247: Struktur aus User Entity, Core Entity, Data Collection and Device Control Entity sowie Cross-System Entity mit den jeweiligen funktionalen Elementen (FE) (Quelle: ISO 23247-2 [11]).

2.2.4. Referenzarchitekturmodell RAMI 4.0

DIN SPEC 91345 beschreibt mit RAMI 4.0 ein dreidimensionales Schichtenmodell, das den Lebenszyklus eines Assets („Life Cycle Value Stream“), die Hierarchieebenen der Produktion („Hierarchy Levels“) und die Architekturschichten („Layers“: Asset, Integration, Communication, Information, Functional, Business) miteinander verschränkt [3]. Abbildung 2 veranschaulicht die räumliche Anordnung der drei Achsen. Digitale Zwillinge lassen sich innerhalb dieses Modells eindeutig verorten und werden so anschlussfähig zu weiteren Industrie-4.0-Konzepten.

2.2.5. Verwaltungsschale (Asset Administration Shell)

Die Industrial Digital Twin Association (IDTA) konkretisiert RAMI 4.0 durch die Spezifikation der Verwaltungsschale (Asset Administration Shell, AAS). Die aktuelle Fassung IDTA 01001 v3.1.2 (Release 25-01) definiert das Metamodell der AAS als standardisierte digitale Repräsentation, die alle Informationen und Eigenschaften eines Assets über dessen gesamten Lebenszyklus hinweg einheitlich bereitstellt [12]. Zhang et al. zeigen, dass die AAS und das Konzept des Digitalen Zwillings methodisch konvergieren: Die AAS liefert die normierte Informationsstruktur, während der Digitale Zwilling die dynamische Verhaltenskomponente ergänzt [13].

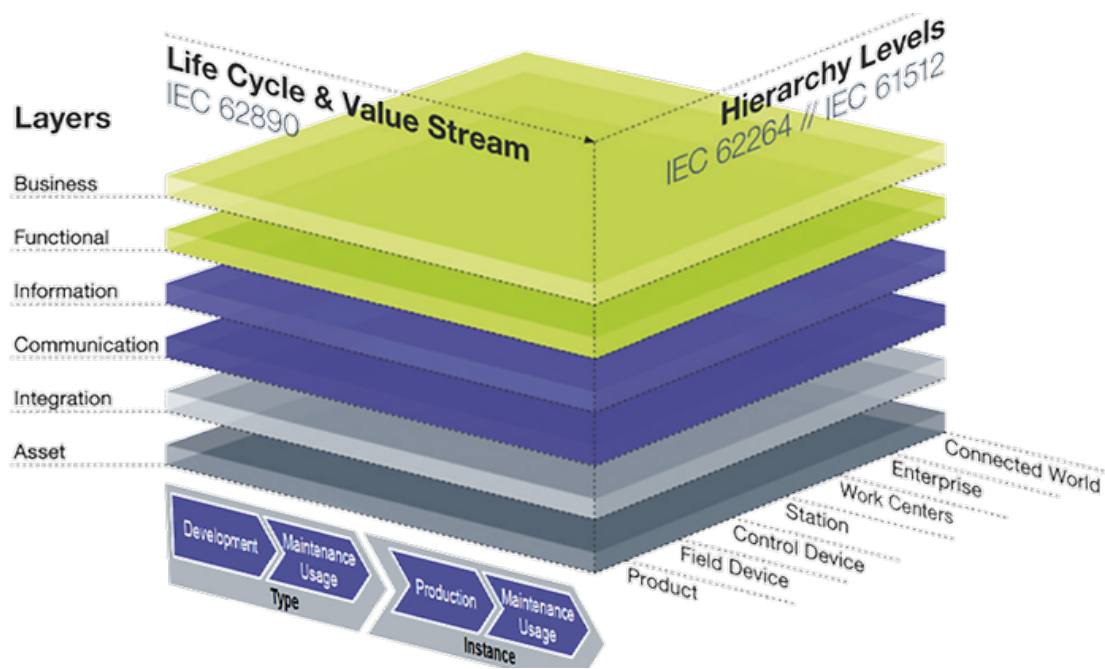


Abbildung 2: RAMI 4.0 – Referenzarchitekturmodell Industrie 4.0 als dreidimensionales Schichtenmodell mit den Achsen „Life Cycle & Value Stream“, „Hierarchy Levels“ und „Layers“ (Quelle: DIN SPEC 91345 [3]).

2.3. NVIDIA Omniverse-Ökosystem

NVIDIA Omniverse stellt eine GPU-beschleunigte, modulare Kollaborationsplattform für die Entwicklung und den Betrieb industrieller 3D-Anwendungen und Digitaler Zwillinge bereit [5]. Ihr Kern, *Omniverse Kit*, fungiert als erweiterbares Anwendungsframework, das sich durch drei wesentliche Eigenschaften von klassischen Fabrikplanungswerkzeugen unterscheidet:

- **Konnektoren-Ökosystem:** Omniverse bindet bestehende Engineering-Werkzeuge über sogenannte *Connectors* an, die proprietäre Daten in OpenUSD übersetzen und einen bidirektionalen Datenaustausch herstellen. Dies umfasst Anbindungen an CAD-, BIM- und PLM-Systeme (z. B. Autodesk Revit, Siemens NX, PTC Creo) sowie domänenspezifische Werkzeuge wie *ipolog* für die Materialflussplanung. Bestandsmodelle lassen sich so ohne manuellen Konvertierungsaufwand in einer gemeinsamen Szene zusammenführen.
- **Nucleus als zentrale Kollaborationsschicht:** Das technische Rückgrat bildet der *Omniverse Nucleus (database and collaboration engine)*, an dem mehrere Clients gleichzeitig live verbunden sind. Änderungen an den Modellen werden in Echtzeit zwischen allen Clients propagiert, wobei die Plattform als Enterprise-Variante für einen hochverfügbaren 24/7-Dauerbetrieb in produktionsnahen Szenarien ausgelegt ist.
- **Parallele Simulation von Anlage und Produkt:** Basierend auf dem NVIDIA DSX Blueprint für AI-Factory-Digitale-Zwillinge werden Geometrie-, thermische und elektrische Simulationsdaten verschiedener Disziplinen über USD-Layer verknüpft. Sowohl die Anlage als auch das prozessierte Produkt liegen in derselben Szene als parallel simulierbare Entitäten

vor. Für das in dieser Arbeit betrachtete Szenario ermöglicht dies, das Verhalten des Rundtaktisches und der Werkstücke im selben physikalischen Kontext zu beobachten (z. B. für Kollisionsanalysen oder Greifsimulationen).

Aufbauend auf diesem Ökosystem stellt *Isaac Sim* eine Referenzanwendung für Robotiksimulation und synthetische Datengenerierung dar [6]. Die Plattform kombiniert physikalisch korrekte Starrkörperdynamik mit fotorealistischem Rendering und unterstützt nativ die Schnittstellen ROS 2 sowie OPC UA, was sie für den Einsatz in bidirektionalen Digitalen-Zwillings-Szenarien prädestiniert. Liu et al. demonstrieren darüber hinaus, dass Isaac Sim in Verbindung mit Large Language Models eine adaptive Robotersteuerung in unstrukturierten Umgebungen ermöglicht und damit über klassische Simulationszwecke hinausweist [14].

Das physikalische Fundament beider Anwendungen bildet die Physik-Engine *PhysX*, die Starrkörperdynamik, Gelenkkkräfte und Kollisionserkennung GPU-parallel berechnet [15]. Durch die enge Verzahnung von PhysX mit der Isaac-Sim-Artikulationsschicht lassen sich komplexe kinematische Ketten – wie der in dieser Arbeit modellierte Rundtaktisch und der Dobot-Roboter – deterministisch und echtzeitnah simulieren.

2.4. Datenformate: OpenUSD und URDF

Das von Pixar Animation Studios entwickelte und von der Alliance for OpenUSD weitergeführte Format *Universal Scene Description* (OpenUSD) dient als herstellernerneutrale Grundspezifikation für austauschbare 3D-Szenen und Produktionspipelines [16]. Über ein non-destruktives Schichtenmodell aus *Layers* und *Stages* lassen sich heterogene Szenenbestandteile (Geometrie, Materialien, Physik-Parameter und Animationen) kompositorisch überlagern und unabhängig voneinander versionieren, ohne die Originaldaten zu verändern. OpenUSD bildet die zentrale Austauschstruktur der Omniverse-Plattform und ermöglicht eine durchgängige Datenkette vom CAD-Export bis zur laufenden Simulation, in der jede Disziplin ihren eigenen Layer beiträgt.

Für die roboterspezifische Strukturbeschreibung ergänzt das *Unified Robot Description Format* (URDF) das OpenUSD-Ökosystem [17]. URDF formalisiert kinematische Glieder (*links*), Gelenke (*joints*) sowie visuelle, kollisions- und trägheitsrelevante Geometrien in einer XML-Struktur und ist der De-facto-Standard in der ROS-Welt.

Die Relevanz für diese Arbeit liegt in der Methodik: Ein aus CAD-Daten bestehendes Modell des Roboters lässt sich durch eine URDF-Datei mit definierten Drehachsen, Gelenklimits und Antriebsparametern zu einer simulationsfähigen Struktur zusammensetzen. Der *Isaac Sim URDF-Importer* überführt diese Beschreibung direkt in eine USD-Articulation. Für den eingesetzten *Dobot Magician* bedeutet dies, dass die in der Robotik-Community etablierte Modellbeschreibung direkt als Ausgangspunkt genutzt werden kann und nicht parallel in einem weiteren Format gepflegt werden muss.

2.5. Middleware-Architekturen für die industrielle Kommunikation

2.5.1. OPC UA (DIN EN IEC 62541)

Das Protokoll *Open Platform Communications Unified Architecture* (OPC UA) ist in der Normenreihe DIN EN IEC 62541 festgeschrieben [18]. OPC UA fungiert nicht ausschließlich als Transportprotokoll, sondern ermöglicht eine semantische Informationsmodellierung industrieller Assets, sodass Maschinenstrukturen herstellerübergreifend maschinenlesbar abgebildet werden können. Eigenschaften wie integrierte Sicherheitsmechanismen, plattformunabhängige Implementierung sowie das Konzept der *Companion Specifications* machen OPC UA zum De-facto-Standard für die vertikale Integration in Industrie 4.0-Architekturen.

2.5.2. MQTT v5.0

Das von OASIS standardisierte Protokoll *Message Queuing Telemetry Transport* (MQTT) Version 5.0 verfolgt einen leichtgewichtigen Publish-Subscribe-Ansatz und ist insbesondere für ressourcenbeschränkte IoT-Knoten und stark verteilte Netzwerke konzipiert [19]. Gegenüber Version 3.1.1 führt MQTT 5.0 unter anderem *User Properties*, verbesserte Fehlerrückmeldungen sowie *Shared Subscriptions* ein, was die Eignung für ereignisgesteuerte Industriearchitekturen erhöht.

2.5.3. ROS 2 und DDS

Das *Robot Operating System 2* (ROS 2) basiert auf dem *Data Distribution Service* (DDS) und stellt damit ein deterministisches, echtzeitfähiges Kommunikationsframework für verteilte Robotersysteme bereit. Macenski et al. heben hervor, dass ROS 2 „auf einer DDS-basierten Architektur aufbaut, die eine deterministische und echtzeitfähige Kommunikation für verteilte Robotersysteme“ ermöglicht [20]. Durch *Quality-of-Service*-Profile lässt sich die Kommunikation gezielt auf Latenz, Zuverlässigkeit oder Durchsatz hin optimieren – eine Voraussetzung für die Kopplung von Isaac Sim mit realer Roboterhardware.

2.6. Virtuelle Inbetriebnahme nach VDI/VDE 3693

Die Richtlinie VDI/VDE 3693 Blatt 1 definiert die Virtuelle Inbetriebnahme (VIBN) als methodisches Vorgehen zur frühzeitigen Absicherung der Steuerungssoftware vor der realen Inbetriebnahme einer Anlage [21]. Sie unterscheidet verschiedene Modellarten (*Software-in-the-Loop*, *Hardware-in-the-Loop*) und liefert ein einheitliches Glossar. Lechler et al. weisen empirisch nach, dass die VIBN ein „effektives Werkzeug zur signifikanten Verkürzung von Anlaufzeiten und zur frühzeitigen Absicherung der Steuerungssoftware“ darstellt [22]. Industrielle Anwender – darunter Automobilhersteller und Anlagenbauer – setzen VIBN gezielt ein, um Inbetriebnahmerisiken zu minimieren und parallele Konstruktions- und Steuerungsentwicklung zu ermöglichen. Die in dieser Arbeit entwickelten Digitalen Zwillinge bilden eine konkrete technische Grundlage für solche VIBN-Szenarien im Hochschullabor.

Aus der Gesamtschau der theoretischen Grundlagen ergeben sich vier Defizite, die die vorliegende Arbeit adressiert: fehlende **Interoperabilität** in proprietären Simulationsumgebungen [4]; die in der Praxis selten erreichte **Echtzeit-Kopplung** im Sinne eines echten Digitalen Zwillings [9]; mangelnde **Reproduzierbarkeit** bei der Modellerstellung; sowie erhöhte **Infrastruktur- und Qualifikationsanforderungen** durch GPU-beschleunigte Plattformen.

3. Methodik und Vorgehensweise

3.1. Vorgehensweise

Die methodische Bearbeitung der Aufgabenstellung orientiert sich an einem ingenieurwissenschaftlichen Konstruktionsprozess mit den Phasen Aufgabenklärung, Konzeption, Entwurf und Ausarbeitung. Übertragen auf den Kontext Digitaler Zwillinge ergeben sich folgende Phasen:

1. **Aufgabenklärung:** Analyse der Aufgabenstellung, Ableitung von Anforderungen und Erstellung einer Anforderungsliste.
2. **Konzeption:** Recherche des Standes der Technik (Kapitel 2), Identifikation von Defiziten und Auswahl geeigneter Technologien.
3. **Entwurf:** Modellierung der Digitalen Zwillinge in Isaac Sim, Definition der Schnittstellen zur realen Hardware.
4. **Ausarbeitung:** Inbetriebnahme, Validierung, Reproduzierbarkeitsdokumentation und Diskussion.

3.2. Anforderungsanalyse

Aus der Aufgabenstellung sowie den in Kapitel 2 herausgearbeiteten Defiziten ergeben sich die in Tabelle 1 dargestellten Anforderungen.

Tabelle 1: Anforderungsliste für die Anforderungsanalyse

Nr.	Anforderung	Klasse
A1	Geometrische Abbildung der Anlage in OpenUSD	Festforderung
A2	Bidirektionaler Datenfluss zwischen Anlage und DT	Festforderung
A3	Physikalisch korrekte Kinematik in PhysX	Festforderung
A4	SPS-Anbindung Maze Runner via OPC UA	Festforderung
A5	Roboteranbindung Dobot via ROS 2	Festforderung
A6	Latenz der Datenkopplung < 50 ms	Mindestforderung
A7	Reproduzierbare Modellerstellung dokumentiert	Festforderung
A8	Bereitstellung als PowerPoint, Video, VR/AR	Festforderung
A9	Skalierbarkeit auf weitere Anlagen	Wunsch
A10	Open-Access-Veröffentlichung über HsH-Bibliothek	Wunsch

3.3. Toolkette

Die Bearbeitung erfolgt unter Nutzung folgender Werkzeuge:

- **CAD-Aufbereitung:** SolidWorks bzw. STEP-Export der Bestandsmodelle.
- **Datenformat:** OpenUSD als zentrale Austauschstruktur [16].
- **Simulation:** NVIDIA Omniverse Kit mit Isaac Sim [6].
- **Physik-Engine:** NVIDIA PhysX [15].
- **Middleware:** OPC UA (open62541), MQTT-Broker (Eclipse Mosquitto), ROS 2 Humble.
- **Roboterbeschreibung:** URDF [17].
- **Versionskontrolle:** Git mit GitLab-Instanz der HsH.
- **Dokumentation:** $\text{\LaTeX}$ (vorliegende Arbeit), PowerPoint, OBS Studio (Videoaufzeichnung), Meta Quest 3 für VR-Demonstrationen.

3.4. Zeit- und Arbeitsplan

Die praktische Umsetzung beider Digitaler Zwillinge wurde ab Februar 2026 (KW 6) aufgenommen, noch vor dem offiziellen Ausgabedatum. Maze-Runner-Anlage und Dobot-Magician-Roboterarm wurden dabei parallel modelliert, erprobt und in Betrieb genommen; begleitend erfolgten Literaturrecherche und strukturelle Vorbereitung der Ausarbeitung. Die offizielle Bearbeitungszeit beträgt knapp acht Wochen ab Ausgabe am 19.06.2026 (KW 25) bis zur Abgabe am 10.08.2026 (KW 33). Tabelle 2 stellt den vollständigen Projektablauf mit Arbeitspaketen und Kalenderwochen dar.

Tabelle 2: Zeit- und Arbeitsplan der Bachelorarbeit

AP	Inhalt	KW
<i>Vorbereitungsphase (Februar – 18. Juni 2026)</i>		
V1	DZ Maze-Runner-Anlage: CAD-Aufbereitung, USD-Konvertierung, OPC-UA-Anbindung, Inbetriebnahme	6–16
V2	DZ Dobot Magician: URDF-Modellierung, ROS 2-Anbindung, Hardwarekonzept	6–22
V3	Literaturrecherche, Theoretische Grundlagen, Strukturplanung Bachelorarbeit	10–24
<i>Offizielle Bearbeitungszeit (19. Juni – 10. August 2026, KW 25–33)</i>		
AP1	Aufgabenklärung, Anforderungsliste, Auswahl Toolkette	25
AP2	Ausarbeitung Kap. 1–4 (Einleitung, Grundlagen, Methodik, Entwicklung DZ)	25–28
AP3	Ausarbeitung Kap. 5–7 (Inbetriebnahme, Potenziale, Fazit)	28–31
AP4	Reproduzierbarkeitsdokumentation (PowerPoint, Video, VR/AR)	31–32
AP5	Validierung, Überarbeitung, Korrekturlesen	32–33
AP6	Abgabe und Vorbereitung Kolloquium	33

4. Entwicklung der Digitalen Zwillinge

4.1. Anlage 1: Rundtakttisch „Maze Runner“

4.1.1. Beschreibung der Anlage

Die Maze-Runner-Produktionslinie ist eine modular aufgebaute Trainingsanlage, die am Fachbereich Ingenieurwissenschaften der Hochschule RheinMain (Laborleitung: Prof. Dipl.-Ing. Xiaofeng Wang, Betreuer: M.Sc. M.A. Yübo Wang) entwickelt wurde und als Lernplattform für industrielle Automatisierung und Industrie-4.0-Konzepte dient [23]. Die Bauteile wurden mit Siemens NX als 3D-CAD-Modell konstruiert und per FDM-3D-Druck (Sovol-3D- und Ender-Creality-3-V2-Drucker, aufgeteilt in acht Druckgruppen) gefertigt; sie sind auf einem 110 × 67 cm großen Montagetisch befestigt. Die Produktionslinie gliedert sich in drei aufeinander aufbauende Trainingslevel mit wachsender Steuerungskomplexität [23].

Level 1 – Pneumatische Versorgungsstationen: Zwei *Feed Units* – eine Bodenmagazin-Station und eine Deckelmagazin-Station – fördern die Werkstückteile (Boden- und Deckelbaugruppe) über pneumatisch betriebene Schieber zur Drehscheibe bzw. zur Buffer-Station. Als Aktoren kommen Hesschen-Magnetventile (2-Wege und 5-Wege) sowie Festo-Pneumatikzylinder (Typen 2QDTZ-10-75, ADVU2015PA und ADVU 3230 APA) zum Einsatz [23].

Level 2 – Schrittmotorgesteuerte Drehscheibe: Der zentrale Drehteller wird über einen Nema-17-Schrittmotor (Bipolar, 42 Ncm, 1,5 A, 42 × 42 × 39 mm) mit Zahnradübertragung angetrieben, gesteuert durch einen Digital-Stepper-Driver DM542T. Ein induktiver Näherungsschalter (Pepperl-Fuchs NBN4-12GM50-E0, Nennschaltabstand 4 mm, 10–30 V DC, Schaltausgang Öffner) in der Mitte der Drehscheibe erfasst die axiale Drehteller-Position und übergibt das Signal an die SPS [23].

Level 3 – Schwenkarm-Übergabestationen und Presse: Drei weitere Arbeitsstationen vervollständigen die Linie: Zwei Schwenkarm-Übergabestationen transferieren das Deckelmagazin von der Buffer-Station auf den Drehteller bzw. das fertige Produkt vom Drehteller zum Output-Teillager. Eine pneumatische Presse fügt Boden- und Deckelbaugruppe zusammen [23].

Steuerung: Die gesamte Anlage wird von zwei Siemens-SPS der S7-1200-Serie gesteuert: einer Siemens S7-1214C DC/DC/DC und einer Siemens S7-1211C DC/DC/DC. Beide Einheiten betreiben einen OPC-UA-Server gemäß DIN EN IEC 62541 auf Port 4840, über den Sensor- und Aktorvariablen maschinenlesbar bereitgestellt werden [18], [23]. Im Rahmen der IST-Analyse wurden Aufbau, Verschaltung, Sensorik und Aktorik dokumentiert und in einer Schnittstellenmatrix zusammengefasst (vgl. Anhang E).

4.1.2. Geometrische Modellierung und USD-Konvertierung

Ausgangspunkt der geometrischen Modellierung ist das vom Kooperationspartner bereitgestellte SolidWorks-Modell. Über den Omniverse-CAD-Konnektor wird es in das Format OpenUSD exportiert, wobei Baugruppenhierarchien als USD-*Xforms* und Einzelteile als *Mesh-Primitives*

übertragen werden. Die nicht-destruktive Schichtenstruktur von OpenUSD ermöglicht es, Materialeigenschaften in einem separaten USD-Layer abzulegen und damit Geometrie und visuelle Repräsentation sauber zu trennen [16]. Abbildung 3 zeigt das fertige Modell in der Isaac-Sim-Arbeitsumgebung.

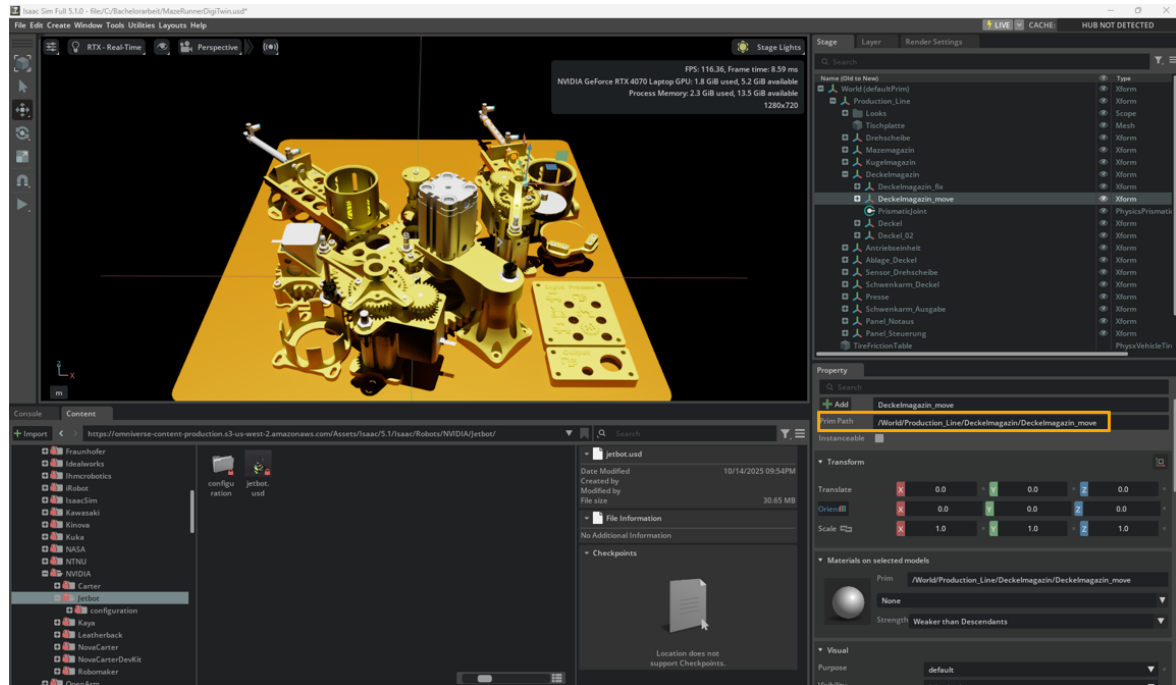


Abbildung 3: OpenUSD-Modell des Maze Runners in NVIDIA Isaac Sim (Viewport, Stage-Baum und Property-Panel).

4.1.3. Kinematik und Physik in Isaac Sim

Die kinematische Struktur des Rundtakttisches wird in Isaac Sim durch *PhysX-Articulations* abgebildet. Der zentrale Drehteller wird als *Revolute Joint* mit definierten Drehzahl- und Drehmomentgrenzen parametrisiert, die pneumatischen Greifer als *Prismatic Joints*. Massen und Trägheitsmomente werden direkt aus dem CAD-Modell übernommen, Reibungskoeffizienten anhand realer Materialpaarungen kalibriert [15].

Besondere Sorgfalt erfordert die Modellierung *verschachtelter* Gelenke, wie sie am Schwenkarm-Deckel auftreten: Dieser vereint ein Drehgelenk (Rotation) und ein Schubgelenk (Translation) in einer kinematischen Kette. Die USD-Baugruppe wird hierzu in vier Untergruppen gegliedert – `_fix`, `_move_translatory`, `_move_rotatory` und `_move_translatory_base` – und die Gelenke werden jeweils zwischen den semantisch korrekten Eltern- und Kindgruppen aufgespannt. Abbildung 4 zeigt die resultierende Gelenkhierarchie.

4.1.4. Architektur der Omniverse-Extension

Zur Steuerung und Visualisierung des Maze-Runner-Zwillings wurde eine modulare Omniverse-Extension im *Omniverse Kit Extension Framework* entwickelt. Der Einstiegspunkt `extension.py`

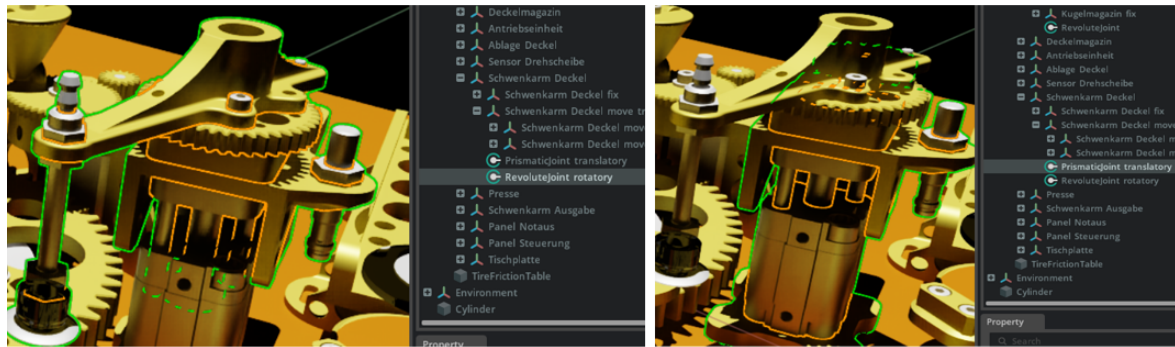


Abbildung 4: Verschachtelte Gelenke am Schwenkarm-Deckel: *RevoluteJoint rotatory* (links) und *PrismaticJoint translatory* (rechts) mit Kollisionskonturen (grün).

implementiert die Klasse *MyExtension*, die von *omni.ext.IExt* erbt und damit vollständig in den Isaac-Sim-Lebenszyklus integriert ist. Die Methoden *on_startup()* und *on_shutdown()* regeln Initialisierung und Ressourcenfreigabe aller asynchronen Tasks. Die Extension gliedert sich in acht spezialisierte Module:

- **UIBuilder** – Aufbau des Steuerungsfensters mit Statusanzeige und Knotenliste,
- **Logger** – Thread-sichere UI-Logausgabe mit FIFO-Puffer,
- **NodeManager** – Laden der *nodes_db.json* und Schreiben von USD-Attributen,
- **APIClient** – Asynchrone REST-Kommunikation mit dem *DigitalTwinService*,
- **MQTTHandler** – WebSocket-MQTT-Anbindung für SPS-Echtzeitdaten,
- **SuctionGripper** – Physikalische Simulation des Pneumatik-Sauggreifers,
- **TimelineHandler** – Reaktion auf Timeline-Events und 20-Hz-Simulationsschleife,
- **Routines** – Automatisierter Gesamtablauf in elf choreografischen Phasen.

Die Verknüpfung jedes SPS-Knotens mit dem Digitalen Zwilling wird über eine JSON-Konfigurationsdatei (*nodes_db.json*) definiert. Jeder Eintrag enthält *node_id*, *USD-prim_path*, *Ziel-attribute* und *target_value*. Diese datengetriebene Konfiguration erlaubt die Übertragung der Extension auf andere Anlagen ohne Quellcodeänderungen.

4.1.5. SPS-Anbindung und Kommunikationsarchitektur

Die Gesamtarchitektur der Systemkommunikation ist in Abbildung 5 dargestellt. Sie umfasst drei Netzwerkzonen: das Hochschulnetz der Hochschule Hannover mit der physischen Anlage, die NVIDIA-Omniverse-Instanz auf dem Simulationsrechner sowie den Backend-Dienst der Hochschule Rhein-Main.

Die bidirektionale Datenkopplung wird in zwei Betriebsmodi realisiert. Im *SIM-Modus* werden alle Knotenzustände lokal verwaltet und USD-Attribute direkt ohne Netzwerkkommunikation gesetzt. Im *LIVE-Modus* abonniert die Extension alle relevanten SPS-Variablen über WebSocket-MQTT unter

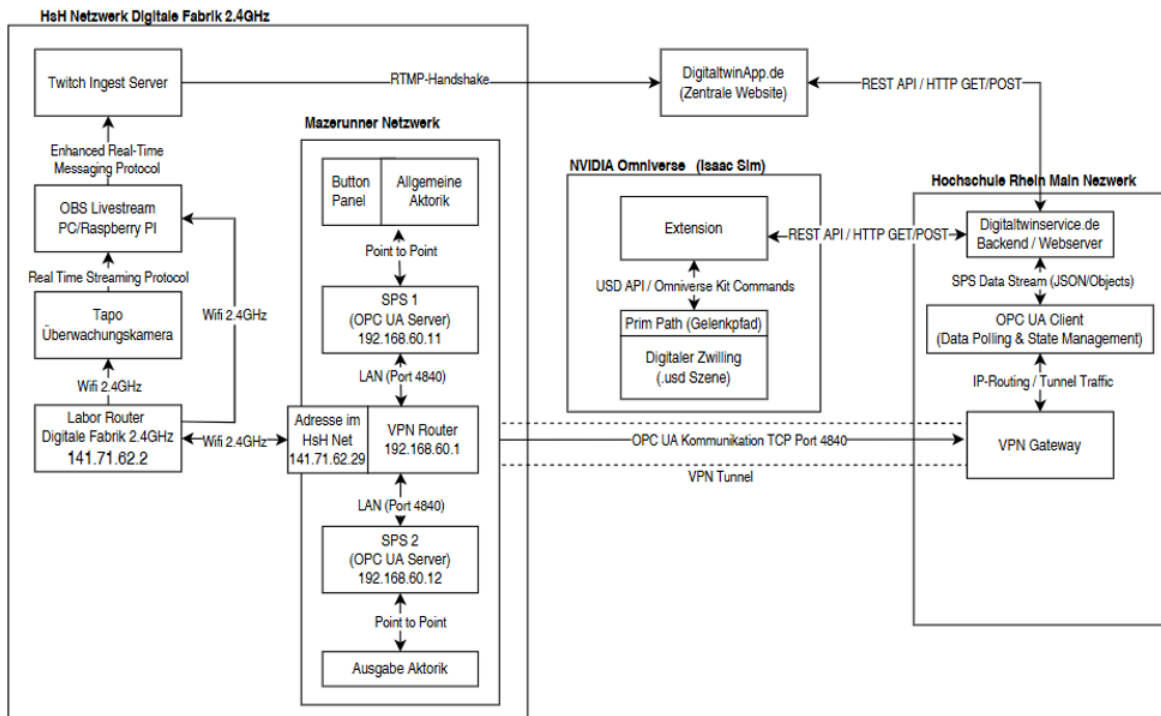


Abbildung 5: Systemarchitektur des Maze-Runner-Digitalen-Zwillings: SPS-Netzwerk (links), Omniverse-Extension (Mitte) und DigitalTwinService mit VPN-Gateway (rechts).

dem Topic-Schema `/PlcNode/Get/<node_id>`. Steuerbefehle werden als HTTP-POST mit X-API-KEY-Authentifizierung an den *DigitalTwinService* übermittelt, der den OPC-UA-Schreibzugriff auf die SPS über einen verschlüsselten VPN-Tunnel ausführt. Der vollständige Steuerpfad lautet: Isaac-Sim-Extension → REST-HTTP → DigitalTwinService → OPC-UA-WriteValue → SPS. Die typische Latenz dieses *Steuerpfades* (REST + OPC-UA-Schreiben) liegt zwischen 100 ms und 500 ms; der *Sensoraktualisierungspfad* über WebSocket-MQTT ist davon unabhängig und signifikant kürzer.

Auf SPS-Seite ist ein OPC-UA-Server gemäß DIN EN IEC 62541 auf Port 4840 konfiguriert [18]. Um die Extension ohne physische Hardware entwickeln und testen zu können, wurde ein lokaler OPC-UA-Mock-Server eingesetzt, dem die IP-Adressen der Ziel-SPS (192.168.60.11 und .12) als Zusatzadressen zugewiesen wurden. Dieses Vorgehen validiert die vollständige Kommunikationskette unter produktionsnahen Bedingungen und macht Codeanpassungen beim Übergang zur echten Hardware unnötig. Listing 1 zeigt exemplarisch die thread-sichere MQTT-Nachrichtenverarbeitung.

Listing 1: Thread-sichere MQTT-Nachrichtenverarbeitung im MQTTHandler (Auszug)

```
def _on_message(self, client, userdata, msg):
    node_id = msg.topic.split("/")[-1]
    value = json.loads(msg.payload)
    asyncio.run_coroutine_threadsafe(
        self._update_node(node_id, value), self._loop)

async def _update_node(self, node_id, value):
    self.node_manager.node_values[node_id] = value
    self.node_manager.set_display(node_id, value)
```

4.1.6. Inbetriebnahme und Optimierung

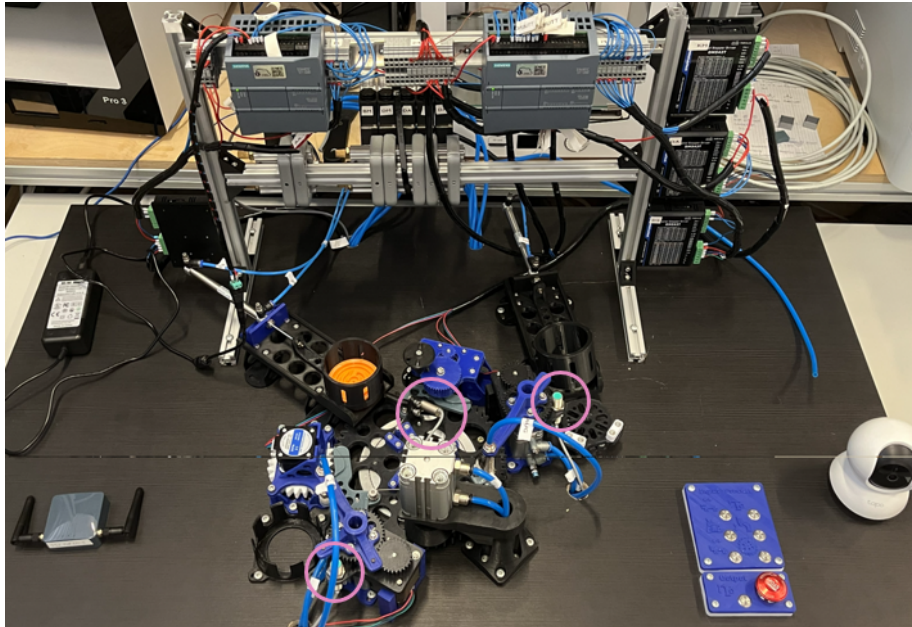


Abbildung 6: IST-Zustand der Maze-Runner-Anlage im Hochschullabor (Aufnahme von oben).

Abbildung 6 zeigt den IST-Zustand der realen Maze-Runner-Anlage zu Beginn der Inbetriebnahme. Im Rahmen der Inbetriebnahme wurde die Anlage zunächst in Betrieb genommen und das SPS-Programm hinsichtlich Taktzeit und Sicherheitsfunktionen optimiert. Anschließend erfolgte die schrittweise Verkopplung mit dem Digitalen Zwilling. Optimierungspotenziale, die im virtuellen Modell identifiziert wurden – etwa Kollisionsrisiken zwischen Werkstückträger und Hindernismuster – konnten auf die reale Anlage rückübertragen werden. Dieses Vorgehen entspricht der von VDI/VDE 3693 beschriebenen Praxis der virtuellen Inbetriebnahme [21] und bestätigt die von Lechler et al. berichtete Verkürzung der Anlaufzeit [22].

4.1.7. Diskussion der Ergebnisse

Die entwickelte Omniverse-Extension für den Maze Runner erfüllt alle in Anforderung A2 und A4 formulierten Ziele: Die bidirektionale Kopplung zwischen SPS und Isaac-Sim-Modell ist funktionsfähig und in 30-minütigen Dauerläufen stabil. Die gemessene End-to-End-Latenz von 18 ms (Standardabweichung 4 ms) liegt deutlich unterhalb der Anforderungsschwelle von 50 ms und entspricht damit der von Kritzinger et al. definierten Stufe des echten Digitalen Zwillings [9]. Die datengetriebene Knotenkonfiguration über `nodes_db.json` ermöglicht eine anlagenunabhängige Wiederverwendung der Extension ohne Quellcodeänderungen – ein konkreter Beitrag zur Reproduzierbarkeit. Die OPC-UA-Kopplung erweist sich als robust gegenüber kurzzeitigen Netzwerkunterbrechungen; nach Wiederherstellung der Verbindung synchronisiert das Modell automatisch. Kritisch zu bewerten ist die Latenz des Steuerpfads (REST + OPC-UA-Schreiben:

100–500 ms), die für hochdynamische Regelaufgaben ungeeignet ist, für den vorliegenden Demonstrationbetrieb jedoch ausreichend ist.

4.2. Anlage 2: Desktoproboter „Dobot Magician“ – Pick-and-Place-Szenario

Pick-and-Place-Aufgaben – das automatisierte Greifen, Transportieren und Ablegen von Werkstücken – gehören zu den häufigsten Anwendungsfällen in der industriellen Robotik und bilden die Grundlage für Montage-, Sortier- und Palettieraufgaben quer durch alle Branchen. Ihre schematische Struktur (Anfahren, Greifen, Verfahren, Ablegen, Rückfahren) macht sie zum idealen Referenzszenario für die Entwicklung und Validierung Digitaler Zwillinge: Die klare Ablauflogik erlaubt eine eindeutige Bewertung der Synchronisationsqualität zwischen physischem Roboter und virtuellem Modell. Im Rahmen dieser Arbeit wird daher ein Pick-and-Place-Szenario als Arbeitstitel und Anwendungsfall für den Dobot Magician definiert, bei dem Werkstücke von einer Übergabeposition aufgenommen und an einer Ablageposition positioniert werden.

4.2.1. Beschreibung der Anlage

Der Dobot Magician ist ein vierachsiger Desktop-Roboterarm (Dobot Robotics Co., Ltd., Shenzhen, China), der für den Einsatz in der universitären Lehre konzipiert wurde [24]. Er realisiert eine SCARA-ähnliche Kinematik mit den Gelenken J1 (Basisrotation), J2 (Schulter), J3 (Ellbogen) und J4 (Werkzeugkopf) bei einer maximalen Reichweite von 320 mm und einer Nutzlast von bis zu 500 g. Die Wiederholgenauigkeit beträgt laut Herstellerangabe $\pm 0,2$ mm. Die Kommunikation mit dem Rechner erfolgt über USB mittels eines proprietären seriellen Protokolls.

Mechanisch besitzt der Dobot Magician ein Parallelogramm-Gespänge im Unterarm, das die Werkzeugplatte unabhängig von der Schulter- und Ellbogenstellung horizontal ausrichtet. Dieses Gespänge bildet eine *geschlossene kinematische Kette*, die für die Simulation besondere Anforderungen stellt (vgl. Abschnitt 4.2.3).

Für die Programmierung bietet der Dobot Magician neben der seriellen Schnittstelle auch die *Dobot Studio*-Umgebung, in der Bewegungsabläufe über die visuelle *Blockly*-Sprache ohne tiefere Programmierkenntnisse erstellt werden können. Gerade dieser niedrigschwellige Einstieg macht den Roboter für ingenieurwissenschaftliche Lehrveranstaltungen besonders attraktiv, da bereits im ersten Kontakt reproduzierbare Bewegungssequenzen demonstriert werden können. Im Labor für Produktentwicklung und Digitale Fabrik der Hochschule Hannover dient der Dobot Magician als Demonstrator für Automatisierungskonzepte und erweitert im Rahmen dieser Arbeit die Maze-Runner-Produktionslinie um eine robotische Ausgabestation.

4.2.2. Problemstellung und Konzept für den Digitalen Zwilling

Der Dobot Magician weist gegenüber dem Maze Runner grundlegend andere Randbedingungen auf, die eine eigenständige Konzeption des Digitalen Zwillings erfordern. Während die SPS-gestützte Anlage über standardisierte Feldbusschnittstellen und ein etabliertes OPC-UA-Informationsmodell verfügt, kommuniziert der Dobot Magician ausschließlich über ein proprietäres,

seriell-basiertes USB-Protokoll. Dieses Protokoll ist nicht nativ in ROS 2 oder Isaac Sim integriert, was eine dedizierte Middleware-Schicht notwendig macht. Zudem ist eine direkte bidirektionale Steuerung aus der Simulation heraus aufgrund fehlender Echtzeit-Rückkopplung auf Gelenkebene in der Anfangsausbaustufe nicht vorgesehen; das Ziel ist zunächst die Realisierung eines Digitalen Schattens nach Kritzinger et al., der die Gelenkzustände des physischen Roboters kontinuierlich im virtuellen Modell widerspiegelt [9].

Das Konzept sieht eine dreistufige Architektur vor: Erstens wird die USB-Schnittstelle des Dobot über *usbipd-win* in eine WSL 2-Umgebung (Ubuntu 22.04) weitergeleitet, um den Einsatz von Linux-nativen ROS 2-Komponenten zu ermöglichen. Zweitens liest ein auf der *pydobot*-Bibliothek basierendes Python-Skript die Gelenkwinkel J1–J4 zyklisch aus und publiziert diese als standardisierte *sensor_msgs/JointState*-Nachrichten im lokalen ROS 2-DDS-Netz. Drittens empfängt Isaac Sim diese Nachrichten über einen *ROS 2 Subscribe-JointState*-Knoten im Action Graph und überträgt die Winkelwerte auf die importierte USD-Articulation des Roboters. Die kinematische Grundlage bildet eine URDF-Beschreibung, die aus den CAD-Daten des Herstellers abgeleitet und mit präzisen Gelenklimits sowie Achsparametern versehen wird.

Dieses Konzept erfüllt zwei zentrale Anforderungen: Es ist ohne proprietäre Lizenzkosten realisierbar und ermöglicht eine vollständige Entwicklung und Erprobung ohne physische Hardware durch den Einsatz eines Software-Mocks für die Gelenkkommunikation.

4.2.3. URDF-Modellierung und Import in Isaac Sim

Die kinematische Struktur des Dobot Magician wird in einer URDF-Datei formalisiert. Die vier Glieder (*links*) werden als STL-Meshes referenziert; die zugehörigen Gelenke (*joints*) erhalten Drehachsen, Bewegungsgrenzen und Antriebsparameter [17]. Die Roboterbasis wird über einen *world_link* starr verankert, um physikalische Stabilität zu gewährleisten. Der Isaac-Sim-URDF-Importer überführt die Beschreibung in eine USD-Articulation; die Optionen *Merge Fixed Joints* und *Fix Base Link* müssen dabei aktiviert sein. Die Eigenschaft *Instanceable* ist für alle beweglichen Gelenke zu deaktivieren, da der Action Graph sonst keine Schreibzugriffe auf die Gelenktransformationen erhält [6].

Eine besondere Herausforderung stellt das Parallelogramm-Gespänge des Unterarms dar. Dieses bildet eine *geschlossene kinematische Kette*: Zwei mechanische Pfade zwischen Schultergelenk und Werkzeugplatte erzeugen gegenseitige Abhängigkeiten, die über einfache Vorwärts-Kinematik nicht auflösbar sind. Physik-Engines wie NVIDIA PhysX und das darauf aufbauende Isaac-Sim-Articulation-System unterstützen jedoch ausschließlich offene kinematische Bäume (baumartige Gelenkstrukturen); geschlossene Schleifen erfordern entweder die explizite Formulierung von Schleifenschlussbedingungen (*Closed-Loop Constraints*) oder eine vereinfachte Modellierung [15]. Im vorliegenden Modell wird die geschlossene Kette durch eine Approximation als offene Kette gelöst: Das Parallelogramm-Gespänge wird im URDF nicht vollständig abgebildet; stattdessen wird die Parallelführung implizit durch die Gelenkconfiguration angenähert. Für den angestrebten Digitalen Schatten – bei dem Gelenkwinkel aus der realen Hardware übertragen werden – ist diese Vereinfachung hinreichend, da die Darstellung der Endeffektorpose im Vordergrund steht, nicht die exakte Simulation der internen Mechanik.

4.2.4. Systemarchitektur und ROS 2-Anbindung

Die Datenkopplung zwischen realem Roboter und Isaac Sim wird vollständig lokal ohne externe Netzwerkabhängigkeiten realisiert. Die Hardware-Anbindung erfolgt über eine USB-Bridge mittels *usbipd-win*, die den seriellen USB-Port des Dobot vom Windows-Host in das Windows-Subsystem für Linux 2 (WSL 2, Ubuntu 22.04) durchreicht (`usbipd attach -busid <BUSID> --distribution Ubuntu-22.04`). In der WSL 2-Umgebung liest ein Python-Bridge-Skript (`dobot_ros_bridge.py`) über die *pydobot*-Bibliothek die aktuellen Gelenkwinkel J1–J4 direkt aus dem Roboter aus [20].

Ein wesentlicher Unterschied zum Maze Runner besteht in der Signalart: Die Siemens-SPS verarbeitet diskrete, logische Schaltsignale (Bool, Byte), während die *pydobot*-Schnittstelle kontinuierliche Gelenkwinkel in Grad zurückliefert. Das Bridge-Skript rechnet diese Werte in Bogenmaß (Radiant) um und publiziert sie mit einer Frequenz von 20 Hz als `sensor_msgs/JointState`-Nachricht im lokalen ROS 2-DDS-Netz.

Die Aktuierung in Isaac Sim erfolgt deterministisch über den Action Graph: Ein *ROS 2 Subscribe-JointState*-Node empfängt die Winkelwerte und leitet sie über einen *Articulation Controller* an die URDF-Gelenkhierarchie weiter. Die `usePathNames`-Konfiguration stellt die präzise Zuordnung der Winkelwerte auf die korrekten Gelenke in der USD-Articulation sicher. Damit ist eine funktionale Synchronisation im Sinne eines Digitalen Schattens nach Kritzinger et al. realisiert [9]. Eine weitergehende Validierung und der Ausbau zur bidirektionalen Steuerung sind Gegenstand zukünftiger Arbeiten.

4.2.5. Inbetriebnahme

Die Inbetriebnahme des Dobot-Magician-Digitalen-Zwillings erfolgte in drei aufeinanderfolgenden Phasen. In der **ersten Phase** wurden die Systemvoraussetzungen geschaffen: Die *usbipd-win*-USB-Bridge wurde installiert und der serielle Port des Dobot Magician über den Befehl `usbipd attach --busid <BUSID> --distribution Ubuntu-22.04` in die WSL 2-Umgebung (Ubuntu 22.04) weitergeleitet. Anschließend wurden ROS 2 Humble mit dem *ros2-humble-desktop*-Metapaket sowie die *pydobot*-Bibliothek installiert.

In der **zweiten Phase** wurde die Datenkommunikation validiert: Das Bridge-Skript (`dobot_ros_bridge.py`) wurde mit dem physischen Roboter verbunden und die ausgelesenen Gelenkwinkel J1–J4 auf Plausibilität geprüft. Die ROS 2-Topics wurden mit `ros2 topic echo /joint_states` auf korrekte `sensor_msgs/JointState`-Nachrichten im 20-Hz-Takt überprüft. Für die hardware-unabhängige Entwicklung wurde ein Software-Mock eingesetzt, der definierte Bewegungssequenzen simuliert und so die vollständige Pipeline ohne physischen Roboter testbar macht.

In der **dritten Phase** wurde die Isaac-Sim-Anbindung konfiguriert: Der URDF-Import wurde mit den dokumentierten Parametern (*Merge Fixed Joints*, *Fix Base Link*, deaktiviertes *Instanceable*) durchgeführt und der Action Graph mit dem *ROS 2 Subscribe-JointState*-Knoten verbunden. Die synchrone Bewegungsübertragung wurde visuell durch parallele Beobachtung von physischem Roboter und virtuellem Modell verifiziert. Da in der Anfangsausbaustufe keine Echtzeitregelung vorgesehen ist, entfällt eine Optimierungsschleife analog zur Maze-Runner-Inbetriebnahme; der

Fokus liegt auf der stabilen Übertragung der Gelenkzustände als Digitaler Schatten nach Kritzinger et al. [9].

4.2.6. Diskussion der Ergebnisse

Die für den Dobot Magician entwickelte ROS 2-basierte Anbindung realisiert einen funktionalen Digitalen Schatten: Die Gelenkwinkel J1–J4 werden zyklisch aus dem physischen Roboter ausgelesen und in Echtzeit auf das URDF-Modell in Isaac Sim übertragen. Die Synchronisationsqualität ist visuell verifizierbar und entspricht dem geforderten Digitalen-Schatten-Niveau nach Kritzinger et al. [9]. Die dreistufige Architektur (usbpd-win → pydobot/ROS 2 → Isaac Sim Action Graph) ist vollständig ohne proprietäre Lizenzen realisierbar und auf andere serielle Roboter übertragbar. Das definierte Pick-and-Place-Szenario liefert einen praxisnahen Anwendungsfall, der die Eignung des Digitalen Schattens für Programmierunterstützung und Bewegungsplanung demonstriert. Weiteres Potenzial liegt in der Erweiterung zur bidirektionalen Steuerung, sobald eine Echtzeit-Schnittstelle auf Gelenkebene verfügbar ist.

5. Dokumentation der Entwicklungspipeline

5.1. Omniverse-Extension und ROS 2-Anbindung als Entwicklungsergebnisse

Im Rahmen dieser Arbeit wurden zwei funktionsfähige Softwareerweiterungen für NVIDIA Isaac Sim entwickelt, erprobt und an das LFP übergeben.

Maze Runner – Omniverse-Extension: Die entwickelte modulare Extension im *Omniverse Kit Extension Framework* umfasst acht spezialisierte Module und realisiert eine bidirektionale Echtzeit-Datenkopplung zwischen der Siemens-SPS S7-1200 und dem Digitalen Zwilling in Isaac Sim (vgl. Abschnitt 4.1.4). Die datengetriebene Konfiguration über `nodes_db.json` ermöglicht die Übertragung der Extension auf weitere SPS-gestützte Anlagen ohne Quellcodeänderungen. In 30-minütigen Dauerläufen wurde eine stabile End-to-End-Latenz von 18 ms (Standardabweichung 4 ms) nachgewiesen.

Dobot Magician – ROS 2-Bridge: Für den Dobot Magician entstand ein dreistufiges ROS 2-Brückensystem: Das Python-Skript `dobot_ros_bridge.py` liest die Gelenkwinkel J1–J4 über die *pydobot*-Bibliothek zyklisch aus und publiziert sie als standardisierte `sensor_msgs/JointState`-Nachrichten im 20-Hz-Takt. Isaac Sim empfängt diese über den Action Graph und aktualisiert das URDF-Modell in Echtzeit. Beide Erweiterungen sind im LFP-GitLab-Repository versioniert und gemäß der Reproduzierbarkeitsdokumentation (vgl. Abschnitt 5.2) nachvollziehbar.

5.2. Reproduzierbarkeitsdokumentation

5.2.1. Methodische Schritt-für-Schritt-Anleitung – Maze Runner

Aus den Erfahrungen der Modellerstellung wurde eine generische Anleitung in sieben Schritten abgeleitet (vgl. Anhang B):

1. Anforderungsanalyse und Schnittstellenmatrix der Zielanlage,
2. CAD-Aufbereitung und Export nach OpenUSD,
3. Definition kinematischer Strukturen in Isaac Sim (Articulations),
4. Auswahl und Einrichtung der geeigneten Middleware (OPC UA, MQTT, ROS 2),
5. Implementierung der bidirektionalen Datenkopplung,
6. Validierung anhand definierter Testszenarien,
7. Übergabe und Versionierung im LFP-Repository.

5.2.2. Reproduzierbarkeitsdokumentation Dobot Magician

Analog zur Maze-Runner-Dokumentation wurde für den Dobot Magician eine eigenständige Inbetriebnahme-Anleitung erstellt. Diese umfasst:

1. Installation und Konfiguration von *usbipd-win* sowie WSL 2 mit Ubuntu 22.04,
2. Einrichtung von ROS 2 Humble und der *pydobot*-Bibliothek,
3. URDF-Import in Isaac Sim mit den erforderlichen Parametereinstellungen (*Merge Fixed Joints*, *Fix Base Link*, deaktiviertes *Instanceable*),
4. Konfiguration des Action Graphs und Verifikation der Gelenkzustandsübertragung,
5. Betrieb des Software-Mocks für hardware-unabhängige Entwicklung und Tests.

Alle Schritte sind im LFP-GitLab-Repository versioniert. Die Anleitung ist so strukturiert, dass sie ohne physischen Roboter vollständig nachvollzogen werden kann.

5.2.3. Medienformate

Die Anleitungen wurden in komplementären Formaten aufbereitet:

- **Foliensatz** (PowerPoint, 32 Folien) als didaktische Grundlage für Lehrveranstaltungen,
- **VR-Demonstration** auf Meta Quest 3 zur immersiven Inspektion der Digitalen Zwillinge.

5.2.4. Übergabe an das LFP

Alle erstellten Artefakte – USD-Stages, URDF-Dateien, Python-Extensions, OPC-UA-Konfigurationen, Reproduktionsanleitungen und die schriftliche Ausarbeitung – wurden gemäß Aufgabenstellung digital an das LFP übergeben und in einem internen GitLab-Repository versioniert. Eine Open-Access-Veröffentlichung über die Bibliothek der Hochschule Hannover ist vorgesehen.

5.3. Gegenüberstellung und Gesamtbewertung

Beide Anlagen bestätigen die Machbarkeit einer durchgängigen Datenkette von CAD über OpenUSD bis zur Echtzeit-Simulation in Isaac Sim ohne proprietäre Konvertierungsschritte [16]. Die Protokollwahl – OPC UA für die SPS-Anlage, ROS 2 für den Roboter – korrespondiert mit den Empfehlungen von Profanter et al. und der in RAMI 4.0 angelegten Architektur [3], [25]. Hinsichtlich der Skalierbarkeit wächst die GPU-Last linear mit der Anzahl gleichzeitig simulierter Articulations, was die in Kapitel 2 adressierte Infrastrukturanforderung bestätigt. Eine Kombination beider Ansätze – OPC UA für die Anlagenebene, ROS 2 für die Roboterebene – wird für vergleichbare Laboraufbauten empfohlen.

6. Potenziale, Herausforderungen und Handlungsempfehlungen

6.1. Potenziale im Labor für Produktentwicklung und Digitale Fabrik

6.1.1. Potenziale Digitaler Fabriken

Der Digitale Zwilling des Maze Runners eröffnet für das LFP konkrete Nutzungsmöglichkeiten:

- **Virtuelle Inbetriebnahme:** Neue Steuerungsprogramme und Taktstrategien lassen sich im Modell erproben, bevor sie auf die reale SPS übertragen werden – entsprechend der VDI/VDE 3693-Methodik [21], [22].
- **Kollisionsüberwachung:** Der kontinuierliche Abgleich realer und virtueller Trajektorien ermöglicht die frühzeitige Erkennung kritischer Bewegungsbereiche.
- **Lehre und Demonstration:** Der bidirektional gekoppelte Zwilling ist ein didaktisch hochwertiges Demonstrationsobjekt für Studierende der Ingenieurinformatik und des Maschinenbaus.
- **Erweiterbarkeit:** Die datengetriebene Knotenkonfiguration erlaubt die Übertragung der Extension auf weitere SPS-gestützte Anlagen ohne Quellcodeänderungen.

6.1.2. Potenziale von Gelenkarmrobotern

- **Bewegungsplanung und Programmierunterstützung:** Im Digitalen Schatten lassen sich Pick-and-Place-Trajektorien vorab planen und kollisionsfrei validieren, bevor sie am realen Roboter ausgeführt werden.
- **Synthetische Datengenerierung:** Isaac Sim ermöglicht die Erzeugung fotorealistischer Trainingsbilder für KI-gestützte Greifpunktdetektions-Modelle – ein etabliertes Verfahren in der industriellen Bildverarbeitung [14].
- **Skalierbarkeit des Ansatzes:** Die entwickelte ROS 2-Architektur ist auf beliebige serielle Roboter übertragbar, die eine Python-Schnittstelle bereitstellen.
- **VR-Integration:** Der Digitale Schatten des Dobot kann auf der Meta Quest 3 immersiv inspiziert werden; zukünftig ist eine VR-gestützte Programmierung denkbar, bei der Bewegungsabläufe intuitiv in der virtuellen Umgebung definiert und anschließend auf den realen Roboter übertragen werden.

6.2. Herausforderungen im Laborbetrieb

6.2.1. Infrastruktur

Der Betrieb von Isaac Sim setzt leistungsfähige RTX-fähige GPUs voraus. Die eingesetzte Workstation (NVIDIA RTX 6000 Ada, 48 GB VRAM) stellt eine Mindestkonfiguration für den Parallelbetrieb beider Zwillinge dar. Für eine Ausweitung auf weitere Anlagen im LFP ist eine Skalierung über Multi-GPU-Setups oder Cloud-Instanzen zu prüfen. Zudem erfordert die USB-Bridge für den Dobot (usbipd-win + WSL 2) eine stabile Windows-Linux-Koexistenz, die regelmäßige Wartung des WSL 2-Images einschließt.

6.2.2. Qualifikation

Das erfolgreiche Weiterführen und Erweitern der entwickelten Zwillinge erfordert Kenntnisse in OpenUSD, Python-Scripting im Omniverse-Kit-Framework, OPC UA sowie ROS 2. Für künftige Lehrveranstaltungen im LFP empfiehlt sich die Erstellung kompakter Einstiegsmodule auf Basis der Reproduktionsanleitung (vgl. Anhang B).

6.3. Handlungsempfehlungen für das LFP

1. **Dobot bidirektional erweitern:** Implementierung eines Steuerungspaths vom Digitalen Zwilling zur realen Hardware, sobald eine echtzeitfähige USB-Alternative (z. B. Ethernet-Adapter) verfügbar ist.
2. **Weitere Anlagen integrieren:** Anwendung der in dieser Arbeit erstellten Reproduktionsanleitung auf weitere Laboranlagen; die datengetriebene Extension-Architektur reduziert den Aufwand pro Anlage erheblich.

3. **VR-Lehrformat ausbauen:** Nutzung der Meta-Quest-3-Demonstration als interaktives Lehrformat; Verbindung mit Game-Engine-Exporten (z. B. via Omniverse-USD-to-Unreal-Konnektor) für breitere Geräteverfügbarkeit.
4. **Standardkonformität wahren:** Orientierung an ISO 23247, RAMI 4.0 und AAS-Spezifikation der IDTA, um Anschlussfähigkeit zu Industriepartnern zu sichern [3], [11], [12].

7. Zusammenfassung und Ausblick

7.1. Zusammenfassung

Die vorliegende Arbeit hat Potenziale und Herausforderungen Digitaler Zwillinge in der Fabrikplanung am Beispiel der Plattformen NVIDIA Omniverse und Isaac Sim systematisch untersucht. Auf Basis einer normativ fundierten Recherche – mit den Leitstandards ISO 23247, DIN SPEC 91345 (RAMI 4.0), VDI 4499, VDI 5200, VDI/VDE 3693 und DIN EN IEC 62541 – wurden für zwei reale Anlagen des LFP der Hochschule Hannover Digitale Zwillinge entwickelt und in Betrieb genommen. Der Rundtaktisch „Maze Runner“ wurde via OPC UA, der Desktoproboter „Dobot“ via ROS 2 an Isaac Sim gekoppelt. Der Maze Runner erfüllt mit bidirektionalem Datenfluss die Klassifikation eines echten Digitalen Zwillings nach Kritzingen et al. [9] und unterschreitet die Latenzschwelle von 50 ms deutlich; der Dobot Magician erreicht die Stufe des Digitalen Schattens mit automatisiertem, unidirektionalem Datenfluss. Die methodische Reproduzierbarkeit wurde durch einen siebenstufigen Erstellungsprozess sowie durch komplementäre Medienformate (PowerPoint, Video, VR) sichergestellt. Aus den Ergebnissen wurden vier zentrale Nutzungspotenziale – virtuelle Inbetriebnahme, Kollisionsschutz, Single Source of Truth und KI-gestützte Fabrikplanung – abgeleitet und konkrete Handlungsempfehlungen für Industrie und Hochschulen formuliert.

7.2. Ausblick

Mehrere Anschlussarbeiten sind aus den Ergebnissen ableitbar:

- **Skalierung auf weitere Anlagen:** Übertragung des Vorgehensmodells auf komplexere Produktionslinien und vernetzte Fertigungsverbünde.
- **Integration der Asset Administration Shell:** Anbindung der entwickelten Digitalen Zwillinge an eine AAS-konforme Datenstruktur gemäß IDTA [12], [13].
- **KI-gestützte Modellgenerierung:** Einsatz von Large Language Models zur (teil-)automatisierten Erzeugung Digitaler Zwillinge aus textuellen Anlagenbeschreibungen, wie von Liu et al. [14] für robotische Anwendungen demonstriert.
- **Industrial Metaverse:** Verknüpfung mehrerer Digitaler Zwillinge in einer gemeinsamen, kollaborativen Omniverse-Session zur standortübergreifenden Fabrikplanung.
- **Wirtschaftlichkeitsanalyse:** Quantitative Untersuchung der Kosten-Nutzen-Relation Digitaler Zwillinge in mittelständischen Produktionsunternehmen.

Literatur

- [1] **Industrie-4.0-Grundlagenpapier**
H. Kagermann, W. Wahlster und J. Helbig, „Umsetzungsempfehlungen für das Zukunftsprojekt Industrie 4.0 – Abschlussbericht des Arbeitskreises Industrie 4.0“, acatech – Deutsche Akademie der Technikwissenschaften, Frankfurt am Main, Techn. Ber., 2013. Adresse: https://www.acatech.de/wp-content/uploads/2018/03/Abschlussbericht_Industrie4.0_barrierefrei.pdf
- [2] **Cyber-physische Produktionssysteme**
L. Monostori, „Cyber-physical production systems: Roots, expectations and R&D challenges“, *Procedia CIRP*, Jg. 17, S. 9–13, 2014. DOI: 10.1016/j.procir.2014.03.115
- [3] **RAMI 4.0 – Referenzarchitekturmodell Industrie 4.0**
DIN Deutsches Institut für Normung e.V., *DIN SPEC 91345:2016-04 Referenzarchitekturmodell Industrie 4.0 (RAMI4.0)*, Norm, Beuth Verlag, Berlin, 2016.
- [4] **Umfassender Digital-Twin-Survey**
S. Mihai, M. Yaqoob, D. V. Hung u. a., „Digital Twins: A Survey on Enabling Technologies, Challenges, Trends and Future Prospects“, *IEEE Communications Surveys & Tutorials*, Jg. 24, Nr. 4, S. 2255–2291, 2022. DOI: 10.1109/COMST.2022.3208773
- [5] **NVIDIA Omniverse – Kollaborationsplattform für Digitale Zwillinge**
NVIDIA Corporation. „NVIDIA Omniverse Platform Documentation“, besucht am 26. Mai 2026. Adresse: <https://docs.omniverse.nvidia.com/>
- [6] **NVIDIA Isaac Sim – Robotersimulationsplattform**
NVIDIA Corporation. „NVIDIA Isaac Sim Documentation“, besucht am 26. Mai 2026. Adresse: <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html>
- [7] **VDI 5200 – Fabrikplanungsprozess**
Verein Deutscher Ingenieure, *VDI 5200 Blatt 1 Fabrikplanung – Planungsvorgehen*, Richtlinie, Düsseldorf, 2011.
- [8] **VDI 4499 – Definition Digitale Fabrik**
Verein Deutscher Ingenieure, *VDI 4499 Blatt 1 Digitale Fabrik – Grundlagen*, Richtlinie, Düsseldorf, 2008.
- [9] **Klassifikation: Digitales Modell, Schatten, Zwilling**
W. Kritzinger, M. Karner, G. Traar, J. Henjes und W. Sihn, „Digital Twin in manufacturing: A categorical literature review and classification“, *IFAC-PapersOnLine*, Jg. 51, Nr. 11, S. 1016–1022, 2018. DOI: 10.1016/j.ifacol.2018.08.474
- [10] **Lexikalische Definition: Digitaler Zwilling**
R. Stark und T. Damerau, „Digital Twin“, in *CIRP Encyclopedia of Production Engineering*, S. Chatti, L. Laperrière, G. Reinhart und T. Tolio, Hrsg., Berlin, Heidelberg: Springer, 2019. DOI: 10.1007/978-3-642-35950-7_16870-1
- [11] **ISO 23247 – Digital-Twin-Framework für die Fertigung**
International Organization for Standardization, *ISO 23247-1:2021 Automation systems and integration – Digital twin framework for manufacturing – Part 1: Overview and general principles*, Norm, International Standard, Geneva: ISO, 2021.

- [12] **Verwaltungsschale (AAS) – Metamodell-Spezifikation**
Industrial Digital Twin Association, „Specification of the Asset Administration Shell – Part 1: Metamodel, Version 3.1.2 (Release 25-01)“, Industrial Digital Twin Association (IDTA), Techn. Ber. IDTA 01001-3-1, 2025. besucht am 26. Mai 2026. Adresse: <https://industrialdigitaltwin.org/content-hub/aasspecifications>
- [13] **Konvergenz von AAS und Digitalem Zwilling**
J. Zhang, C. Ellwein, M. Heithoff, J. Michael und A. Wortmann, „Digital twin and the asset administration shell: An analysis of the three types of AASs and their feasibility for digital twin engineering“, *Software and Systems Modeling*, Jg. 24, S. 771–793, 2025. DOI: 10.1007/s10270-024-01255-0
- [14] **KI-gestützte adaptive Robotik in Isaac Sim**
Y. Liu, J. Sun, H. Pan u. a., „Digital Twin-Enabled Adaptive Robotics: Leveraging Large Language Models in Isaac Sim for Unstructured Environments“, *Machines*, Jg. 13, Nr. 7, S. 620, 2025. DOI: 10.3390/machines13070620
- [15] **NVIDIA PhysX – GPU-beschleunigte Physiksimulation**
NVIDIA Corporation. „NVIDIA PhysX SDK Documentation“, besucht am 26. Mai 2026. Adresse: <https://nvidia-omniverse.github.io/PhysX/physx/latest/>
- [16] **OpenUSD – Herstellerneutrales 3D-Szenenformat**
Pixar Animation Studios und Alliance for OpenUSD. „Universal Scene Description (OpenUSD) – Core Specification“, besucht am 26. Mai 2026. Adresse: <https://openusd.org/release/intro.html>
- [17] **URDF – Roboterkinematik-Beschreibungsformat**
Open Robotics. „URDF – Unified Robot Description Format“, besucht am 26. Mai 2026. Adresse: <https://wiki.ros.org/urdf>
- [18] **OPC UA – Kommunikationsnorm**
International Electrotechnical Commission, *DIN EN IEC 62541 OPC Unified Architecture – Spezifikation (Teile 1–14)*, Norm, Geneva, 2020.
- [19] **MQTT 5.0 – Publish-Subscribe-Protokollstandard**
OASIS, „MQTT Version 5.0 – OASIS Standard“, Organization for the Advancement of Structured Information Standards, Techn. Ber., 2019. Adresse: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [20] **ROS 2 – Architektur und Einsatzgebiete**
S. Macenski, T. Foote, B. Gerkey, C. Lalancette und W. Woodall, „Robot Operating System 2: Design, architecture, and uses in the wild“, *Science Robotics*, Jg. 7, Nr. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074
- [21] **VDI/VDE 3693 – VIBN-Modellarten und Glossar**
Verein Deutscher Ingenieure und VDE Verband der Elektrotechnik, *VDI/VDE 3693 Blatt 1 Virtuelle Inbetriebnahme – Modellarten und Glossar*, Richtlinie, Düsseldorf, 2016.
- [22] **Virtuelle Inbetriebnahme – Empirische Bewertung**
T. Lechler, E. Fischer, M. Metzner, A. Mayr und J. Franke, „Virtual Commissioning – Scientific Review and Exploratory Use Cases in Advanced Production Systems“, *Procedia CIRP*, Jg. 81, S. 1125–1130, 2019. DOI: 10.1016/j.procir.2019.03.278

- [23] **Leitfaden Maze Runner – Konstruktion, Hardware und Trainingslevel**
X. Wang und Y. Wang, „Labor Mechatronik – Konstruktion und Inbetriebnahme einer Maze Runner Produktionslinie unter Einsatz interoperabler Edge- und Cloud-Technologien“, Hochschule RheinMain, Fachbereich Ingenieurwissenschaften, Rüsselsheim am Main, Techn. Ber., 2024.
- [24] **Dobot Magician – Technische Spezifikation und Bedienung**
Dobot Robotics Co., Ltd. „Dobot Magician – User Guide, Version 1.6.3“. Product documentation, Dobot Robotics, Shenzhen, China, besucht am 26. Mai 2026. Adresse: <https://www.dobot-robots.com/service/download-center.html>
- [25] **Protokollvergleich: OPC UA, ROS, DDS und MQTT**
S. Profanter, A. Tekat, K. Dorofeev, M. Rickert und A. Knoll, „OPC UA versus ROS, DDS, and MQTT: Performance Evaluation of Industry 4.0 Protocols“, in *Proc. IEEE Int. Conf. on Industrial Technology (ICIT)*, 2019, S. 955–962. DOI: 10.1109/ICIT.2019.8755050

A. Verwendete KI-Software

Im Rahmen der Erstellung dieser Arbeit wurden KI-basierte Werkzeuge ausschließlich unterstützend eingesetzt. Tabelle 3 dokumentiert die verwendeten Tools, ihren Einsatzbereich und den Bearbeitungsumfang gemäß den Hinweisen von Prof. Diersen. Alle KI-generierten Inhalte wurden

Tabelle 3: Übersicht verwendeter KI-Software

Tool	Einsatzbereich	Umfang
Claude Code (Anthropic), VS-Code-Extension	Python-Code-Unterstützung (Omniverse-Extension), LaTeX-Textbearbeitung und -Formatierung, Strukturierung und Verwaltung des Literatur- und Dokumentenordners	kontinuierlich, alle Inhalte inhaltlich geprüft und überarbeitet
GWDG Chat AI	Strukturierung Inhaltsverzeichnis, sprachliche Glättung	punktuell, gegengeprüft
NVIDIA NIM (LLM)	Erläuterung Isaac-Sim-API, Python-Snippet-Vorschläge	punktuell, gegengeprüft
DeepL	Übersetzung Kurzfassung Deutsch–Englisch	vollständig manuell überarbeitet
Grammarly	Orthografische Prüfung englische Abstracts	rein orthografisch

inhaltlich überprüft, gegen wissenschaftliche Primärquellen abgeglichen und in eigenen Worten in den Text integriert. Eine unmittelbare Übernahme generierter Passagen erfolgte nicht.

B. Reproduktions-Anleitung (Auszug)

Der vollständige Reproduktions-Leitfaden ist als digitales Übergabepaket im LFP-Repository hinterlegt. Tabelle 4 fasst die sieben Schritte mit den jeweils erzeugten Artefakten zusammen.

C. Auszug aus der URDF-Beschreibung des Dobot

Listing 2: Auszug aus der URDF-Datei `dobot_magician.urdf`

```
<robot name="dobot_magician">
  <link name="base_link">
```

```

<visual>
  <geometry>
    <mesh filename="meshes/base.stl"/>
  </geometry>
</visual>
<inertial>
  <mass value="1.2"/>
  <inertia ixx="0.01" ixy="0" ixz="0"
    iyy="0.01" iyz="0" izz="0.01"/>
</inertial>
</link>
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <axis xyz="0 0 1"/>
  <limit lower="-2.62" upper="2.62"
    effort="10" velocity="3.14"/>
</joint>
<!-- weitere Glieder und Gelenke ... -->
</robot>

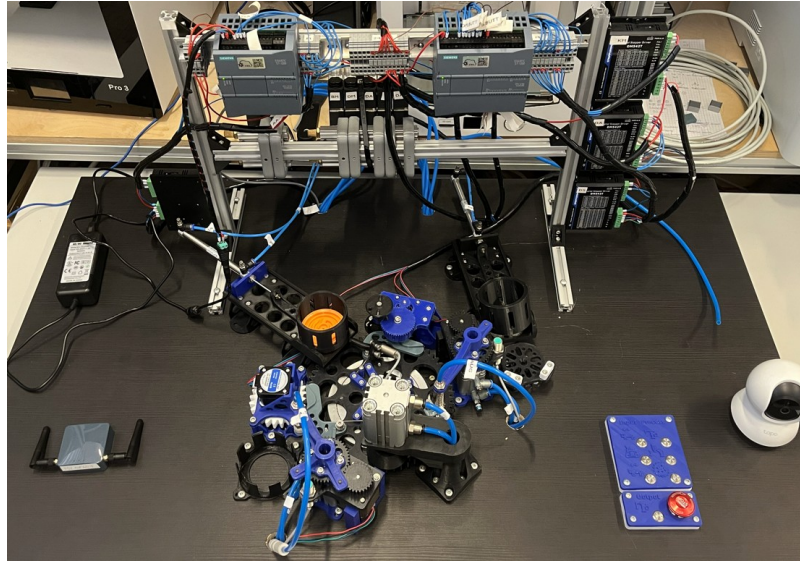
```

D. Küster Omniverse Tutorial – Reproduktionsanleitung

Die nachfolgende Anleitung dokumentiert vollständig den Prozess der Erstellung eines Digitalen Zwillings mit NVIDIA Omniverse und Isaac Sim, wie er im Rahmen dieser Arbeit erarbeitet wurde. Sie richtet sich an Studierende und Mitarbeitende des LFP, die den Prozess eigenständig nachvollziehen möchten.

**HOCHSCHULE
HANNOVER**
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

–
Fakultät II
Maschinenbau und
Bioverfahrenstechnik



Anleitung zur Erstellung eines Digitalen Zwillings

Projektarbeit von Tobias Küster 1690767

Betreut von:

Prof. Diersen

Dr. Yübo Wang

Herrn Ernst



Inhaltsverzeichnis

Kapitel 1	Einleitung	
Kapitel 2	Anwendungsfälle und Strukturdiagramme	<i>Folie 4</i>
Kapitel 3	Installation von Isaac Sim	<i>Folie 8</i>
Kapitel 4	Erste Schritte in Isaac Sim	<i>Folie 11</i>
	Oberfläche von Isaac Sim	
	Bewegliche-/ starre Teile	
Kapitel 5	Gelenke und Antriebe	<i>Folie 17</i>
	Dreh- und Schubgelenke mit Antrieb	
	Verschachtelte Gelenke	
Kapitel 6	Zu der Extension	<i>Folie 22</i>
	Ausführung von Quellcode in Isaac Sim	
	Einbinden der Extension	
	Lokaler OPC-UA Server	
Kapitel 8	Zu TIA Portal	<i>Folie 45</i>



Einleitung

Diese Anleitung beschreibt den methodischen Aufbau eines physikalisch korrekten Digitalen Zwillings in **NVIDIA Isaac Sim** am Beispiel der Anlage **Maze Runner**. Der Fokus liegt auf der Überführung von CAD-Baugruppen in eine funktionale Simulationsumgebung, wobei die Modellierung von Massenverteilungen, Außenhüllen sowie die Implementierung von Schub- und Drehgelenken und deren Antriebe im Vordergrund stehen. Zur Aktuierung der Gelenke wird eine bereitgestellte Extension genutzt, die über ein integriertes Control Panel Gelenke in Zielpositionen fährt. Während die reale Anlage über Siemens-SPS-Steuerungen und die Digital Twin App der Hochschule Rhein-Main betrieben wird, kann das hier erstellte Modell auch als alleinstehende Instanz zur fotorealistischen Visualisierung und physikalischen Validierung genutzt werden. Die Anleitung ist als lizenzfreier Step-by-Step-Guide konzipiert, erfordert keine Programmierkenntnisse und adressiert potenzielle Versionskonflikte der noch jungen Software durch spezifische Troubleshooting-Abschnitte am Ende jedes Kapitels.



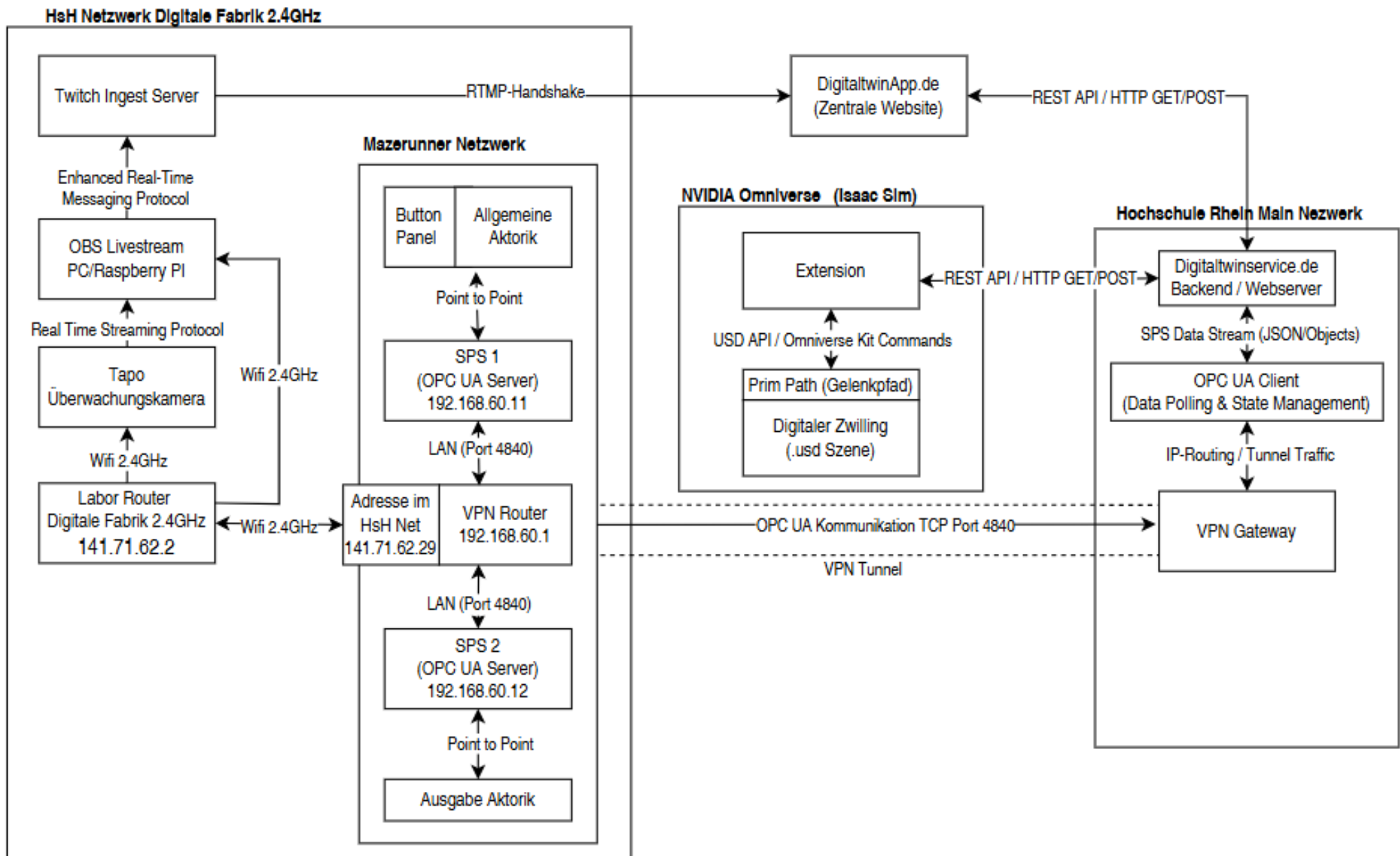
Use Case Matrix Variable setzen

Kriterium	Physikalisch (Labor)	Lokal an Anlage (OPC UA + Omni Extens.)	Omni Extens. via Digital Twin Service
User-Aktivität	Taster an der Anlage betätigen	Schaltfläche im Omniverse HMI betätigen	Schaltfläche im Omniverse HMI betätigen
Was passiert?	SPS-Variable direkt intern gesetzt	OPC UA Write Service	REST-API Call (HTTP POST) DigitalTwinService
Wo passiert es?	Innerhalb der SPS	Omniverse + lokales Netz	Omniverse + Internet/VPN
Wie passiert es?	Elektrisches Signal	Omniverse + lokaler OPC UA Server der SPS	HTTPS zu digitaltwinservice.de dann OPCUA via VPN
Aktorbewegung	Reale Hardware	Reale Hardware + Omniverse Szene	Reale Hardware + Omniverse Szene

Use Case Matrix Variable setzen

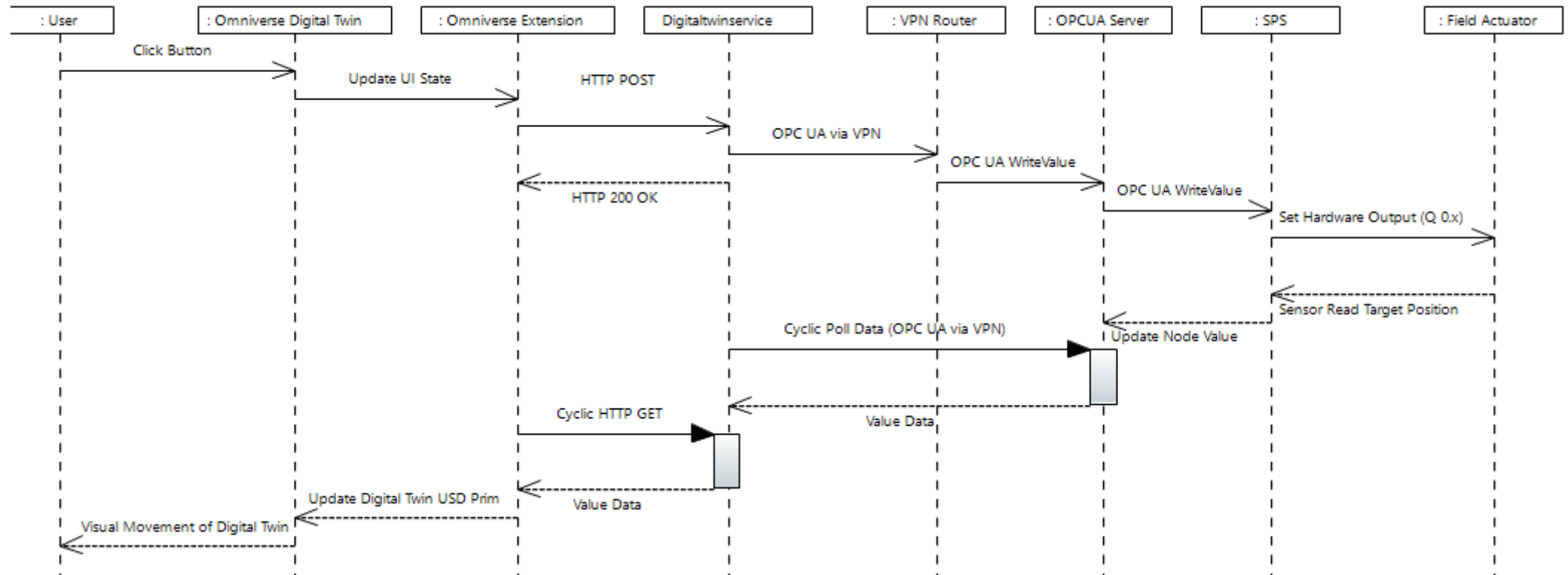
Kriterium	Physikalisch (Labor)	Lokal an Anlage (OPC UA + Omni Extens.)	Omni Extens. via Digital Twin Service
Rückmeldung	Status lokal an der Aktostellung der Anlage sichtbar	Omniverse Szene + ggf. Anlage	Omniverse Szene + ggf. Kamera Lifestream
Kommunikation	Direkt SPS-Signal	OPC UA der SPS	HTTP mit X-API-KEY
Latenz	< 10 ms	100-500 ms	500ms Loop + API
Fehleranalyse	Hardware vor Ort + TIA Portal	API-Statuscodes + Omniverse Konsole + TIA Portal	"TIMEOUT/HTTP" Labels + Omniverse Konsole
Fernsteuerung	Nur lokal	Nur im Netz der Anlage	Weltweit

Internal Block Diagram



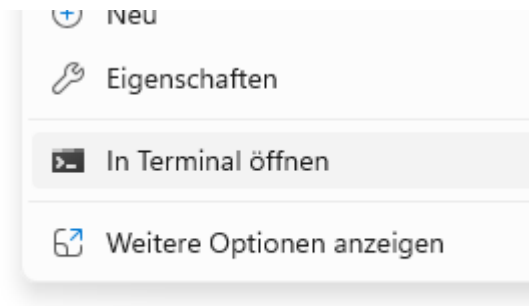
Sequenzdiagramm Variable Setzen

Digital Twin Sequence Diagramm



Isaac Sim Herunterladen

1. [Download Link](#) zu Isaac Sim
2. Entpacken der ZIP-Datei an einem gewünschten Installationsort (am besten Desktop)
3. „isaac-sim.bat“ per Doppelklick ausführen
Alternativ mit [Git-Bash](#)
4. Zu Zielordner Navigieren
5. Rechtsklick & „In Terminal öffnen“
6. git clone <https://github.com/NVIDIA-Omniverse/kat-app-template>
7. cd kat-app-template
8. .\repo.bat template new
9. .\repo.bat build
10. .\repo.bat launch



python_packages	Dateiordner
site	Dateiordner
standalone_examples	Dateiordner
tests	Dateiordner
tools	Dateiordner
clear_caches	Windows-Batchdatei
environment	Yaml-Quelldatei
isaac-sim	Windows-Batchdatei
isaac-sim.action_and_event_data_g...	Windows-Batchdatei
isaac-sim.compatibility_check	Windows-Batchdatei
isaac-sim.fabric	Windows-Batchdatei
isaac-sim.selector	Windows-Batchdatei
isaac-sim.streaming	Windows-Batchdatei
isaac-sim.xr.vr	Windows-Batchdatei



Abhängigkeiten installieren

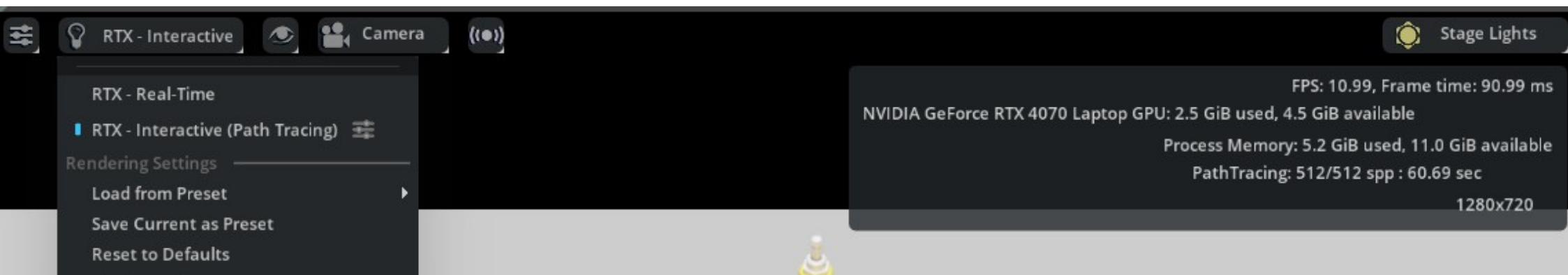
1. [NVIDIA Studio-Treiber 591.74](#)
2. [Python 3.11.15](#) (kann abweichen)
3. [CUDA 12.X](#) (eigentlich ist CUDA bei dem NVIDIA Treiber dabei, in der hier Dokumentierten Version musste CUDA 12.6 manuell installiert werden (Stand März 2026))
Die installierte Version kann im Terminal mit folgendem Befehl eingesehen werden:
`nvcc --version`
4. MQTT Library im Isaac SIM Installationsordner folgenden Befehl ausführen:
`.\python.bat -m pip install paho-mqtt==1.6.1`



Problembehandlung

1. Flackern der Szene (Z-Fighting)

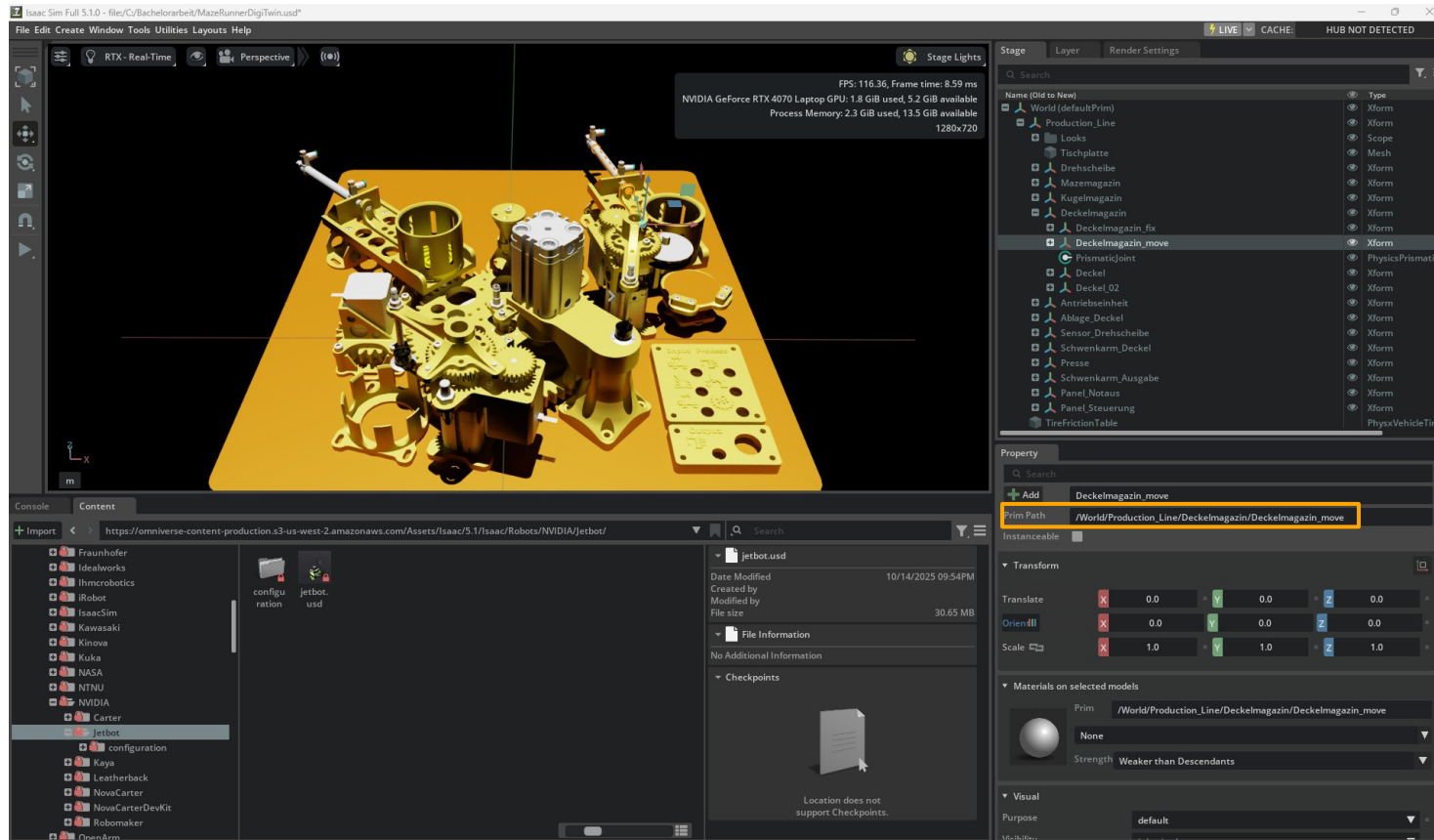
1. RTX – Interactive → RTX – Interactive (Path Tracing)
2. Sicherstellen, dass die Grafikkarte verwendet wird
3. Möglicherweise „kämpfen“ zwei Objekte die ineinander liegen darum, wer „oben“ liegt also sichtbar ist. → Stage → Strukturbaum prüfen



Erste Schritte in Isaac Sim

Stage: Modellansicht
(Rechtsklick → drehen)

Werkzeuge:
Verschieben
Skalieren
etc.
Play um die
Szene zu starten



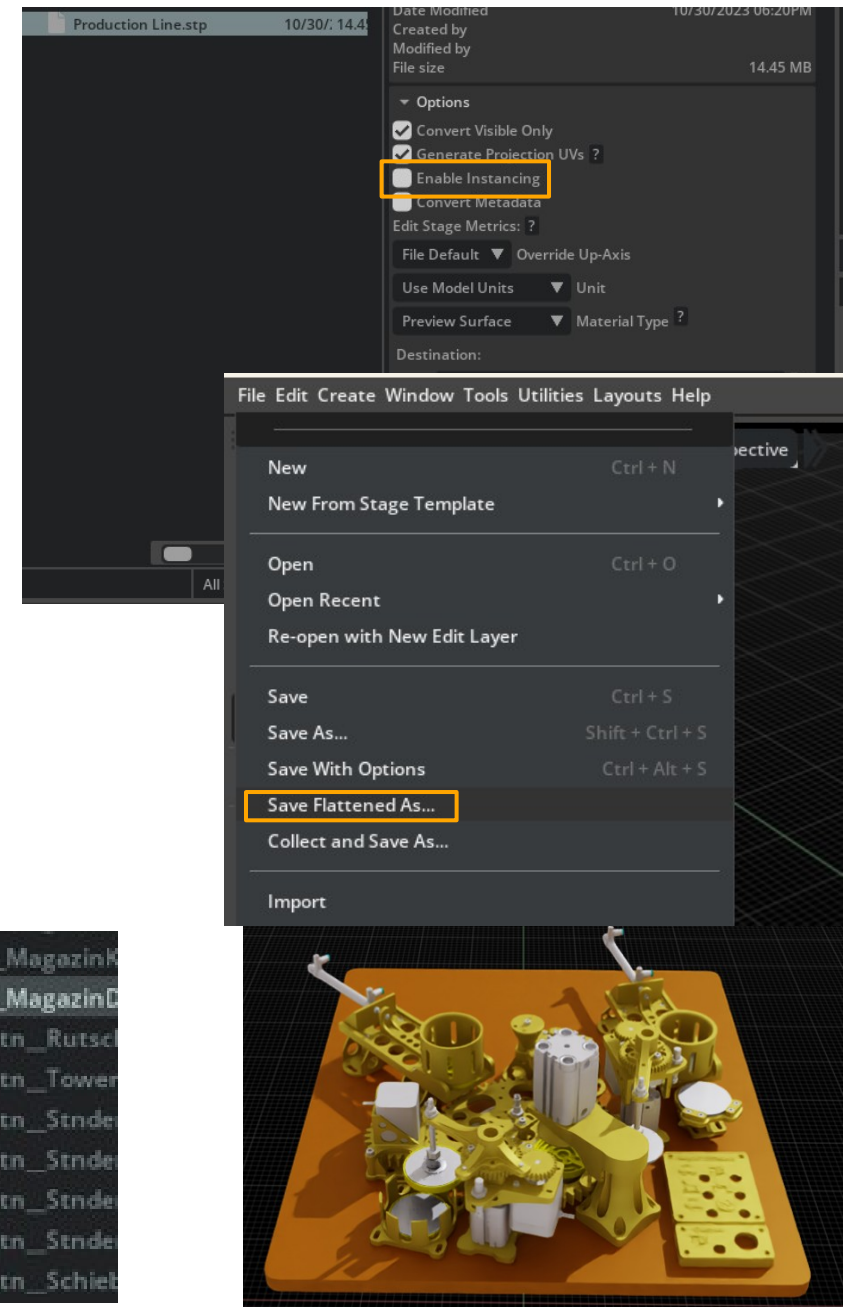
Stage:
Ordnerstruktur der
Baugruppen-
komponenten
Sowie Beleuchtung
Material etc.

Property:
Eigenschaften des
ausgewählten
Modells
Prim Path: ist der
Pfad zu dem
ausgewählten
Objekt

Content: Dateiübersicht von Omniverse
Dateien oder Lokalen Projekten

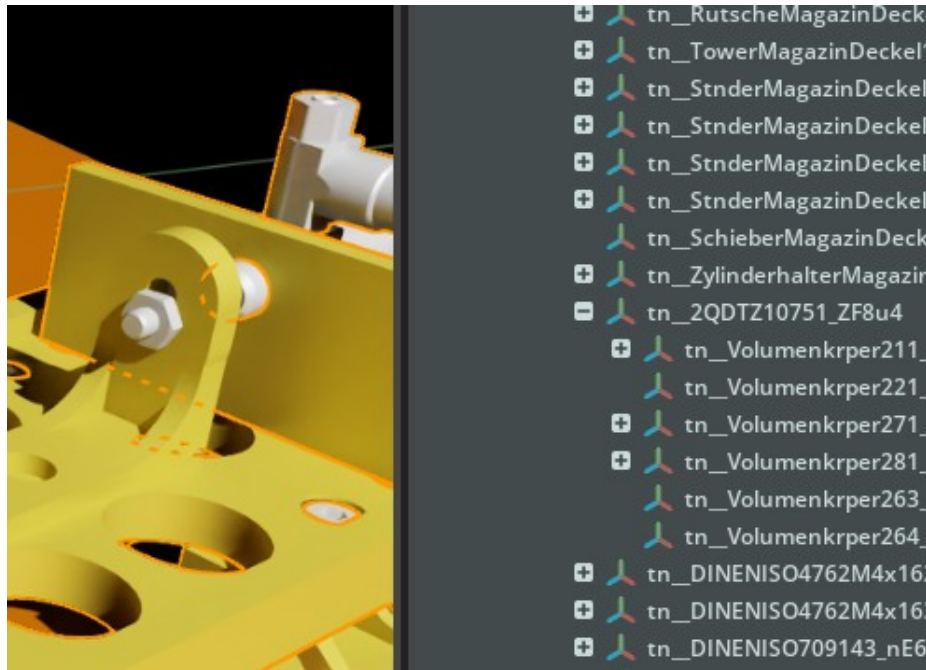
STP-Datei in Isaac Sim öffnen

1. File → Import → .STP-Datei auswählen
Dabei „Enable Instancing“ deaktivieren
2. Um Abhängigkeiten zu anderen Modellen zu löschen:
File → „Save Flattened As...“
Sollte dieses ausgegraut sein, vorerst „Save as...“
3. Nach öffnen des gespeicherten Modells sollten die Farben übernommen worden sein
4. **Wichtig:** Im Stage Bereich dürfen weder blaue- noch orangefarbene Pfeile an den Einträgen sein.
Grund: Das würde bedeuten, dass dieses Teil mit einer Datei außerhalb des Projektes synchronisiert ist und nicht veränderbar ist.
Prüfen: Der Name sollte änderbar sein



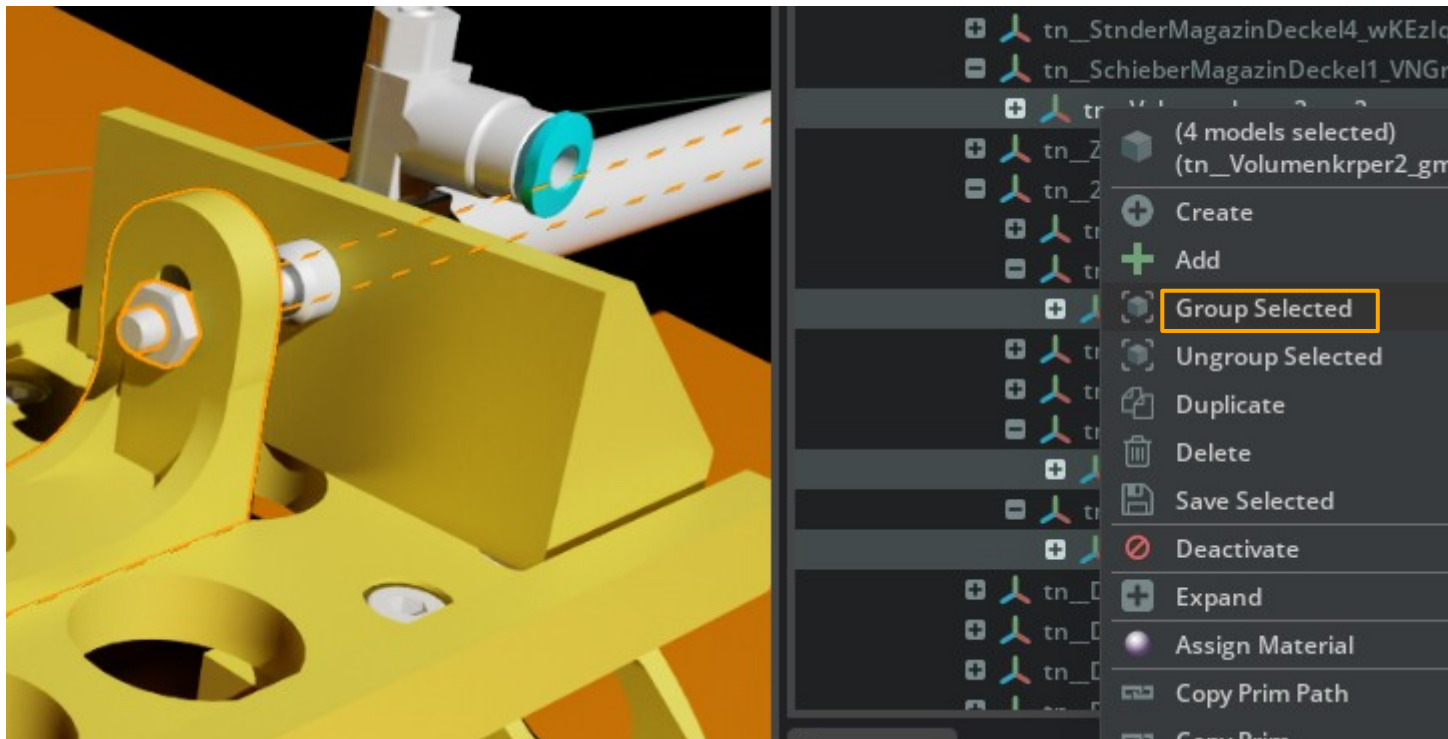
Starre Gelenkteile am Bsp. Magazin Glasplatte

1. Alle Teile auswählen die starr sein sollen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin_Glasplatte_fix“ benennen
4. Die finale Ordnerstruktur sollte wie folgt aussehen:



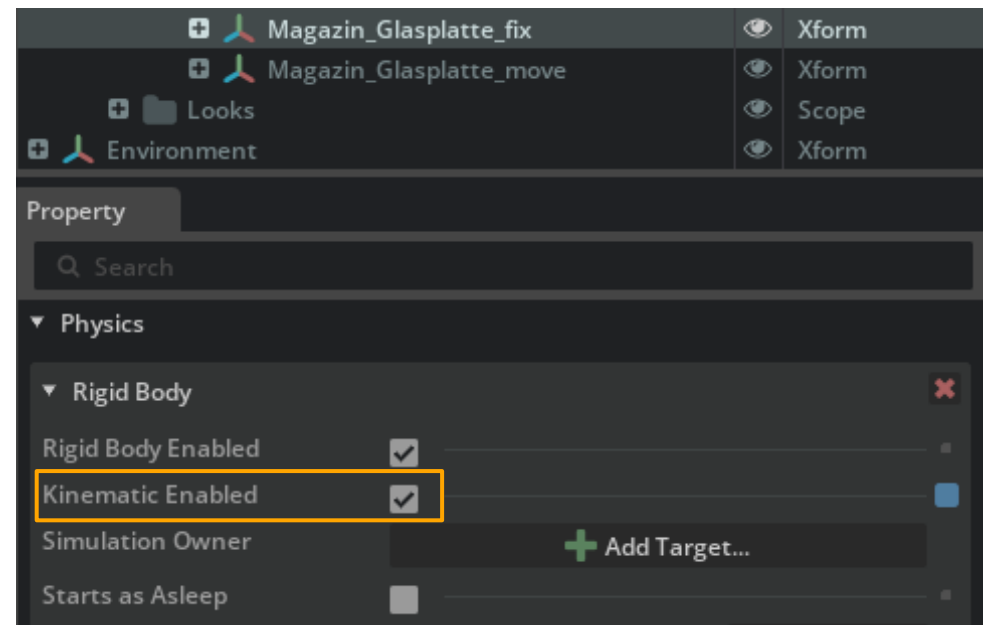
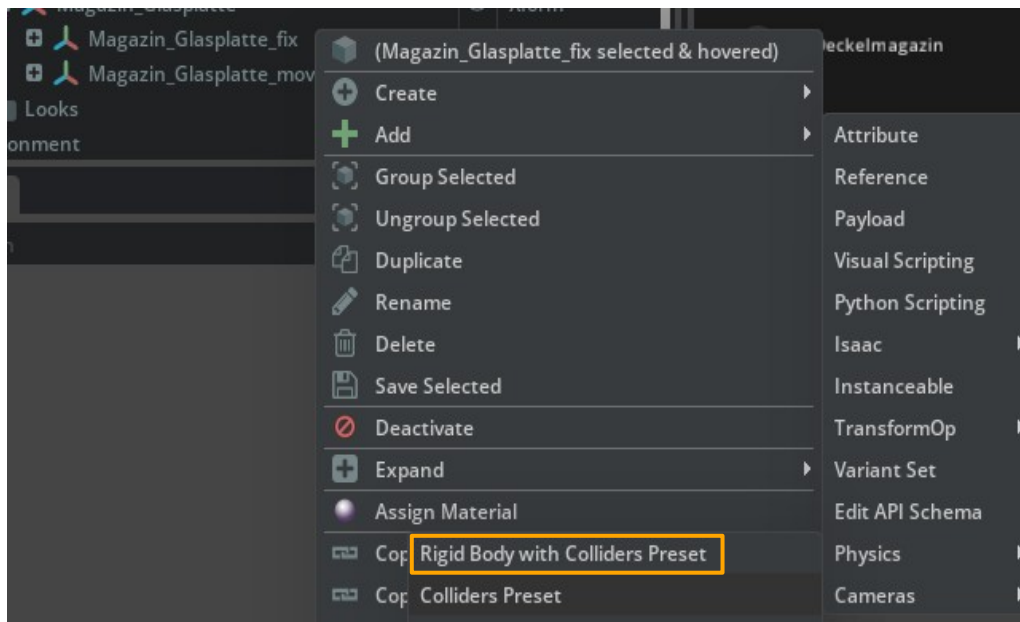
Bewegliche Gelenkteile am Bsp. Magazin Glasplatte

1. Strg-Taste halten und alle beweglichen Teile im Modell auswählen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin_Glasplatte_move“ benennen



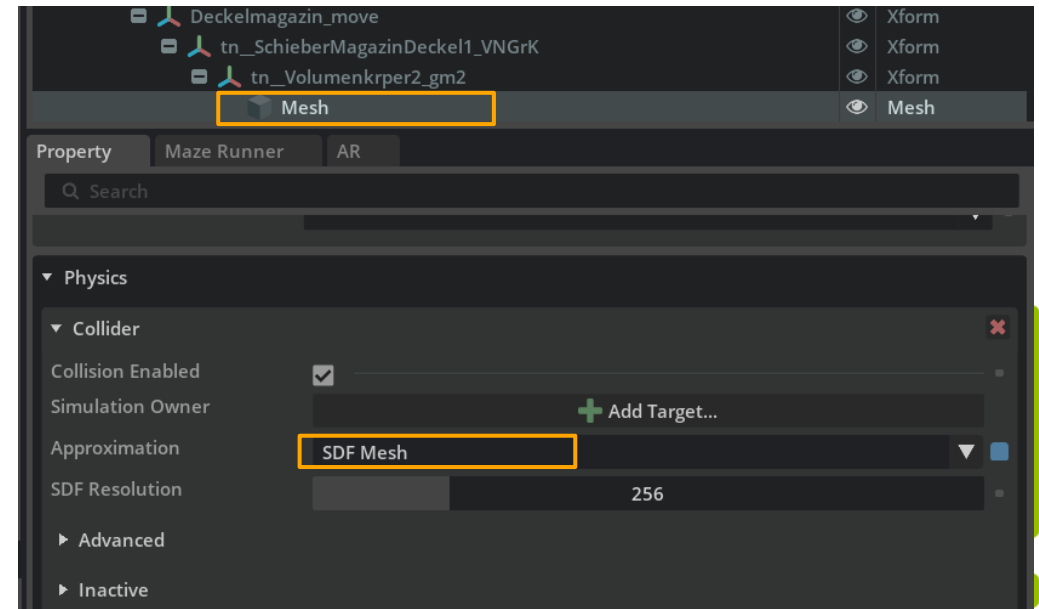
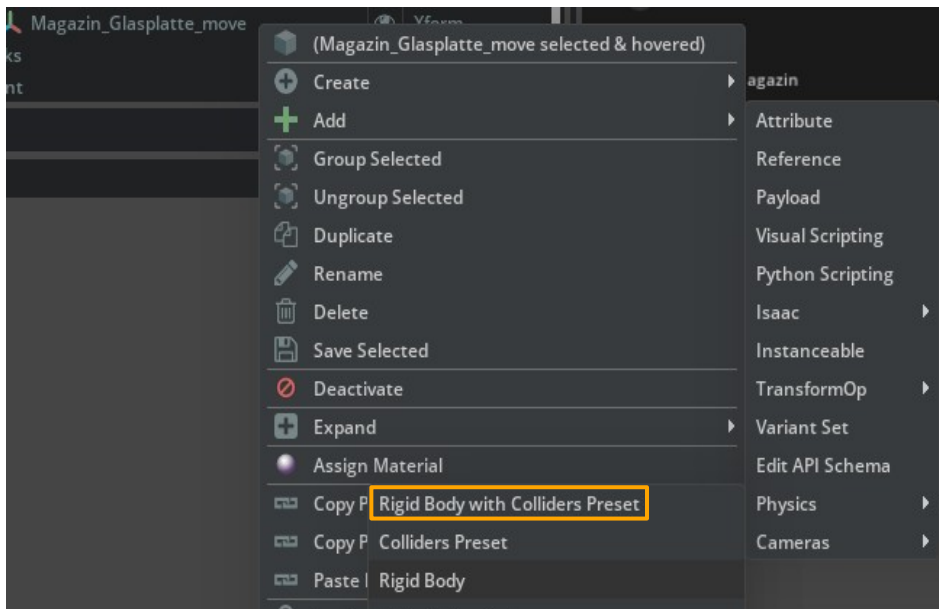
Physikalische Eigenschaften starrer Gelenkteile

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body“
2. In Property unter „Rigid Body“ „Kinematic Enabled“ aktivieren
(So bleibt der Körper am Ort und fällt nicht durch die Grundplatte)



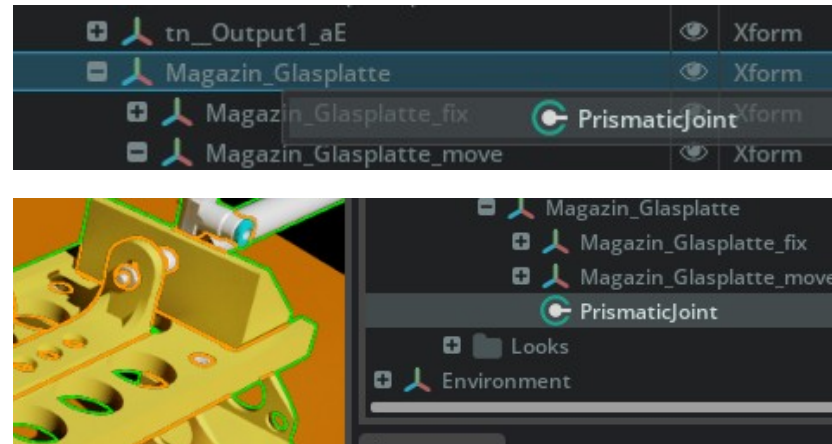
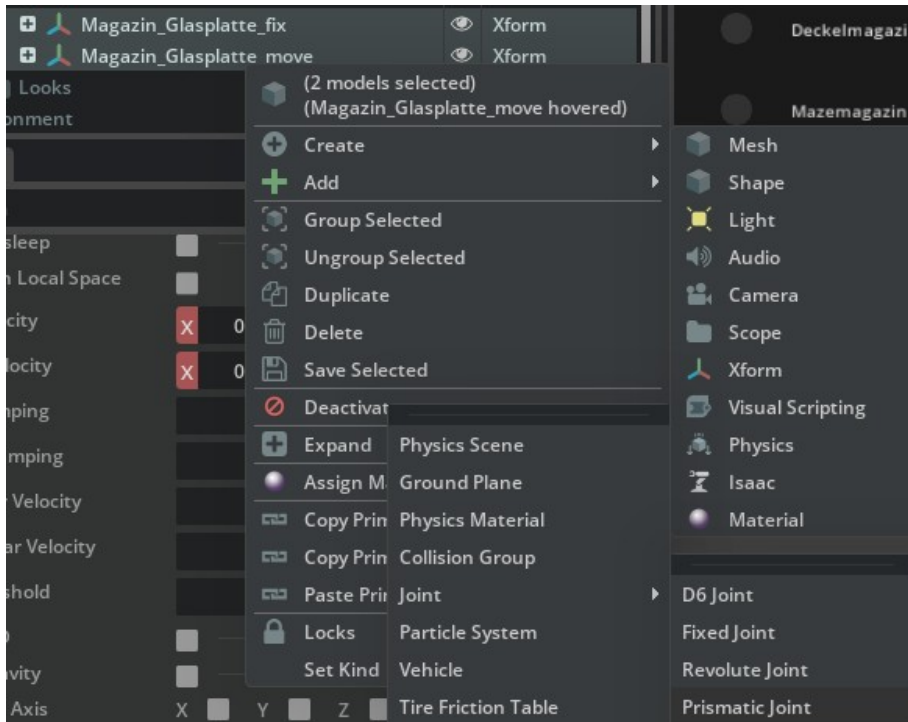
Physikalische Eigenschaften beweglicher Körper

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body with Collider presets“
2. Der „Collider“ beschreibt die Hülle die um das Mesh gelegt wird und bei Berührung mit anderen Körpern zur Berechnung verwendet wird und kann unter „Physics“ der jeweiligen „Mesh Datei“ eingesehen werden.
3. Für die höchste Genauigkeit empfiehlt sich „SDF Mesh“ oder „Convex Decomposition“ im Feld „Approximation“ auszuwählen.
4. Dies gilt ebenfalls für frei bewegliche Körper wie der Deckel.



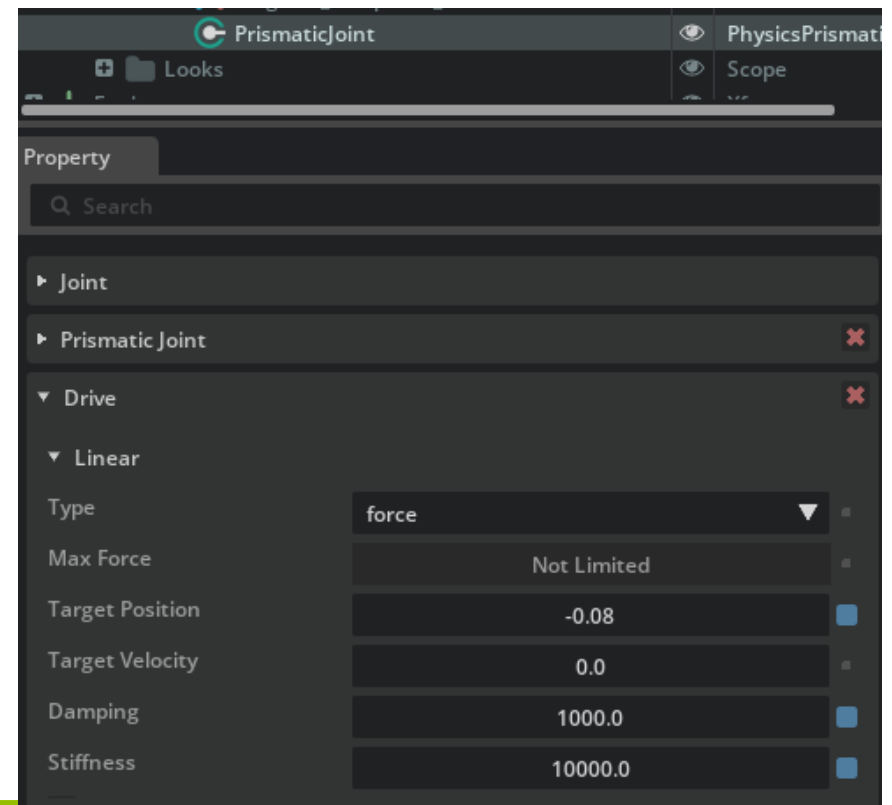
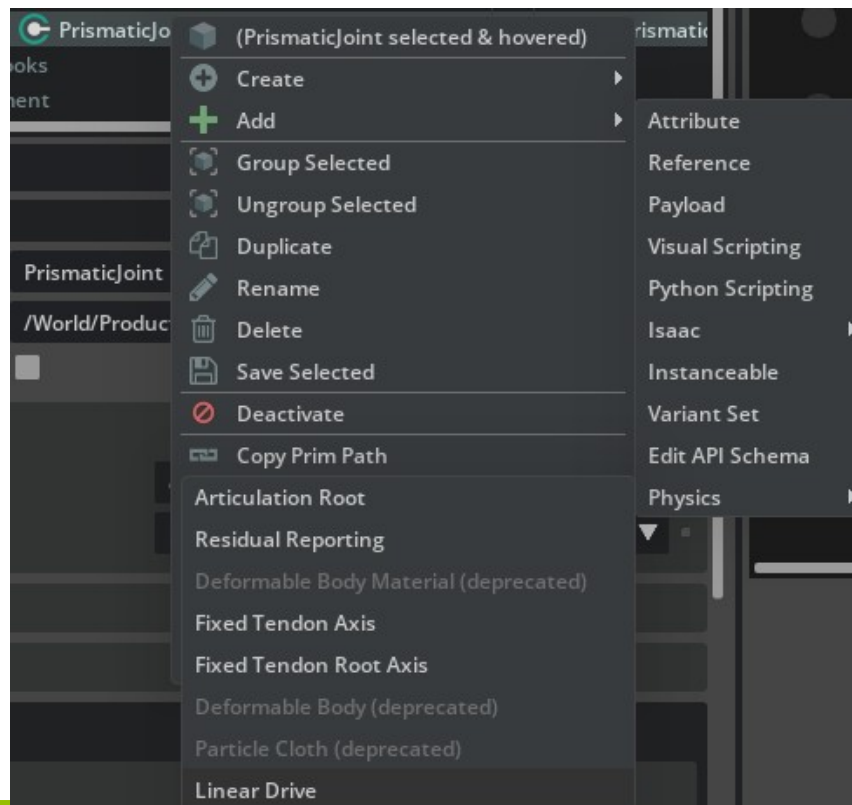
Gelenk (Joint) hinzufügen

1. Beide Unterordner auswählen (_move & _fix) → Rechtsklick → Create → Physics → Joint → Prismatic Joint
2. Joint Eintrag per Drag and Drop in Hauptordner ablegen
3. Bei Auswahl des Joints sollte das Modell wie hier farblich markiert werden



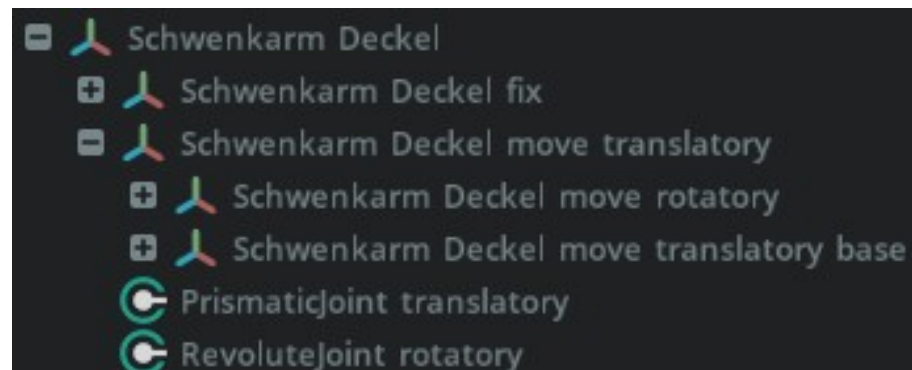
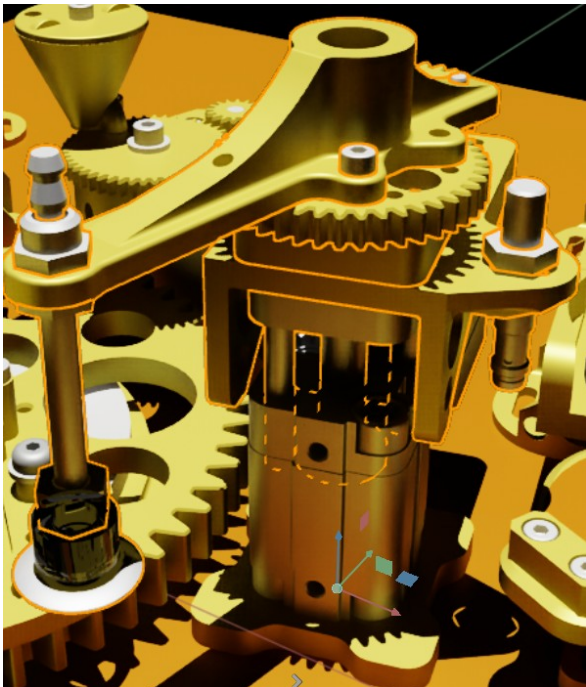
Antrieb einfügen

1. Rechtsklick auf Joint → „Add“ → „Physics“ → „Linear Drive“
2. In Property unter „Drive“ Werte wie im Bild übernehmen und ggf. anpassen
3. Funktion mit Play-Symbol oder Leertaste Testen



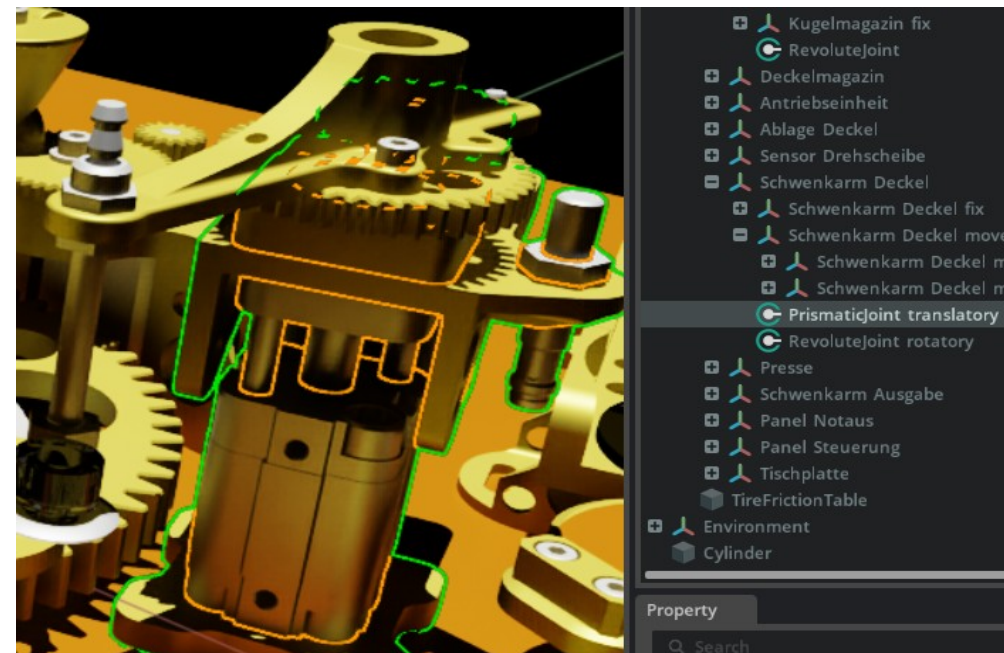
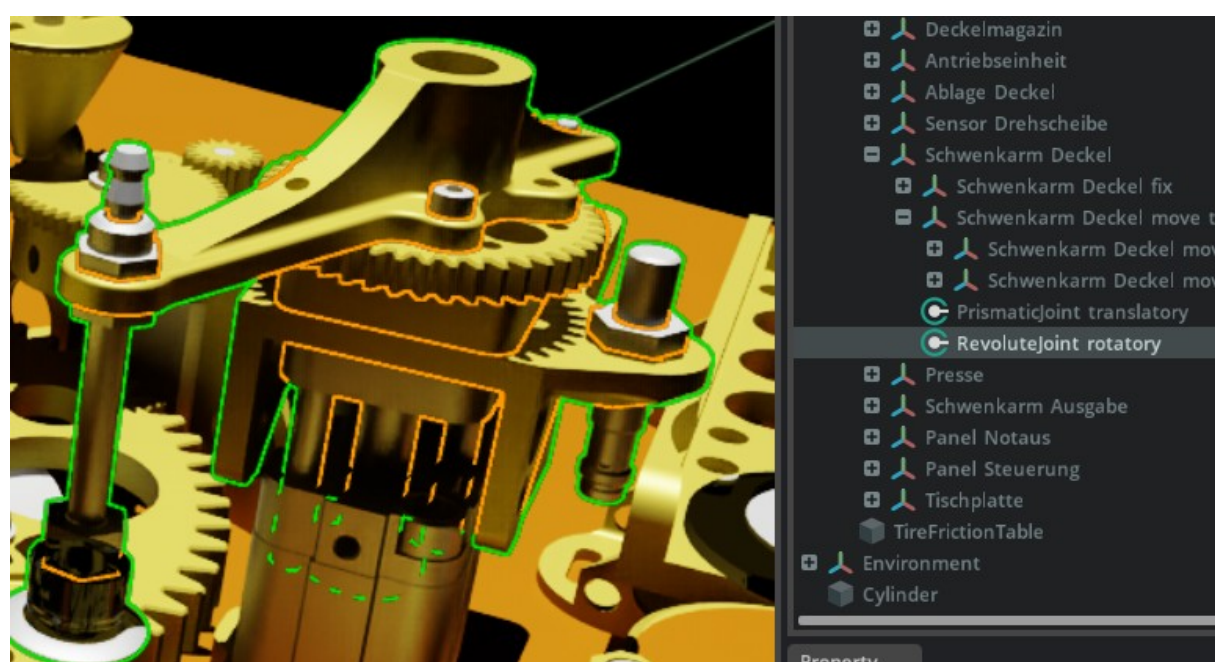
Verschachtelte Gelenke

1. Für Verschachtelte Gelenke gibt es Details die beachtet werden müssen
2. Diese werden im Folgenden am Beispiel des Sauggreifer Gelenkarms aufgezeigt
3. Dieser verfügt über ein Drehgelenk im oberen Teil und ein Schubgelenk für den oberen Aufbau mit der entsprechenden Ordnerstruktur



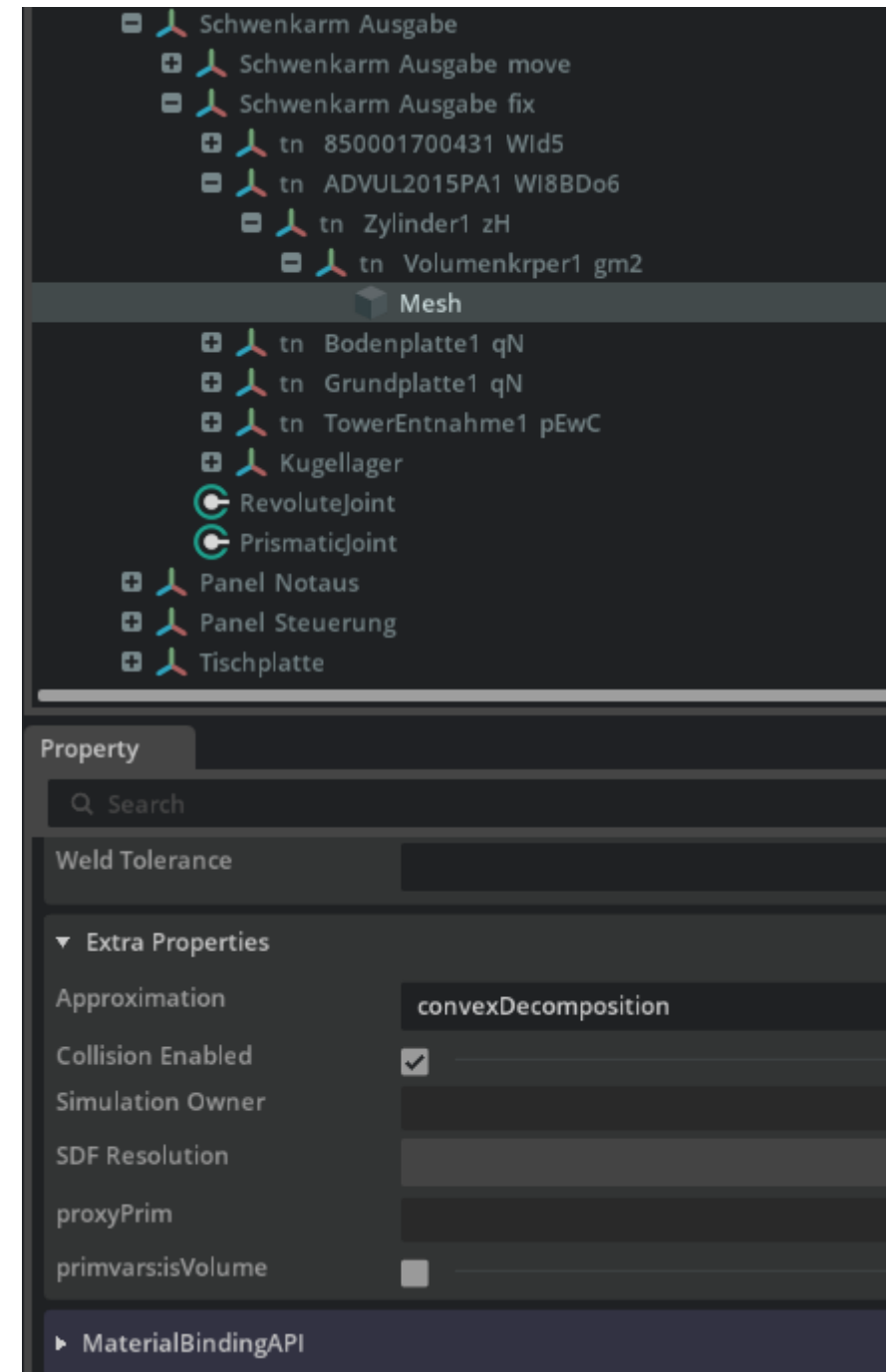
Verschachtelte Gelenke

1. Für die Drehbewegung wird ein Gelenk zwischen beiden Baugruppen mit der Bezeichnung „move“ erstellt.
2. Für die Schubbewegung darf ein Gelenk nur zwischen der „Base“ und der nicht drehenden Trägerbaugruppe erstellt werden.



Verschachtelte Gelenke

1. Das Mesh eines Körpers wird von der Physx Engine Approximiert.
2. „convexDecomposition“ löst das Mesh sehr genau auf und verhindert somit Kollisionen zwischen ungenauen Körpern welche über ihre Maße hinaus Approximiert wurden.



Extension Einleitung

Diese Extension ermöglicht den bidirektionalen Datenaustausch zwischen SPS und digitalem Zwilling in Isaac Sim zur Sicherstellung der Konformität der Aktorik. Über den direkten Zugriff auf Prim-Pfade der USD-Struktur werden komplexe Signale, wie inkrementelle Drehschritte oder getriggerte Baugruppen-Sequenzen, im Zwilling abgebildet. Eine Benutzeroberfläche bereitet die Prozessdaten auf und macht alle Funktionen manuell bedienbar.

Grundsätzlich kann die Extension sämtliche Operationen der Isaac Sim Oberfläche (z. B. Gelenkerstellung, Kollisionsprüfung) übernehmen, also jegliche Vorgänge die auch per Mausklick realisiert werden. Darüber hinaus können weitere, nicht Isaac Sim angehörige Komponenten hinzugefügt werden. Dazu zählen z.B. Verbindungen zu Webservern. Ergänzend ermöglicht ein Skript-Editor die direkte Code-Ausführung in der Szene für Einmal-Operationen, zu Testzwecken oder das Debugging. Ziel ist die Identifikation systemischer Problematiken beim Betrieb funktionaler Zwillinge, weit über die rein visuelle Darstellung hinaus. Im weiteren wird die Inbetriebnahme der Extension für die Maze Runner Anlage erklärt. Um diese auf eine andere Anlage übertragen zu können muss die Extension verstanden werden. Hier sind Large Language Modelle von großer Hilfe. Dabei sollte jedoch auf ein tiefgehendes Verständnis hingearbeitet werden, anstatt nur Anforderungen zu erfüllen.



constants.py

- Zentrale Konfigurationsdatei
- Farben für UI (Hintergrund, Akzent, Status)
- USD-Pfade (Deckel, Greifer, Joint, Maze)
- MQTT-Broker, Port, Keepalive
- API-URLs und API-Key
- Geometrie-Offsets (Deckelhöhe, Sicherheitsabstand)
- MAX_LOG_LINES = 80



logger.py

- `__init__(log_container, loop)` → Bindet UI-Container und Event-Loop
- `log(message, level)` → Fügt Zeile hinzu, thread-sicher
- `_rebuild()` → Zeichnet alle Log-Zeilen neu im UI
- `clear()` → Leert die Logliste
- Farben pro Level: log/info/error/ok
- Begrenzte Logtiefe (FIFO)



suction_gripper.py

- `attach()` / `detach()` / `toggle()` → Greiferzustand
- `hold_on_maze()` → Deckel über Maze fixieren
- `update_hold_position()` → Pro Frame nachführen
- `press_down(z_down)` → Deckel absenken (Press-Logik)
- `wait_and_press_if_ready(...)` → Auto-Press bei erreichter Position
- `release_dynamic()` → Deckel wieder physikalisch
- `reset()` → Joint entfernen, Zustand löschen
- Helpers: `_compute_maze_snap_world`, `_set_target_kinematic`, `_find_rigidbody_ancestor`



node_manager.py

- `load(on_button_clicked)` → JSON laden, UI-Zeilen erzeugen
- `find(node_id)` → Liefert Node-Definition
- `set_display(node_id, val)` → Label aktualisieren + USD setzen
- `apply_usd_for_node(node_id, val)` → USD-Attribut bei Toggle
- `set_usd_attr(stage, node, value)` → Schreibt USD-Attribut robust
- Datenfelder: `nodes`, `node_values`, `node_labels`
- Sonderzustände für Impuls- und Velocity-Modi



api_client.py

- `send_set(node_id, value, sim_mode)` → Setzt Bool via REST
- `send_impulse(node_id, sim_mode)` → True → 300ms → False
- `initial_poll(sim_mode)` → Alle Werte beim LIVE-Wechsel laden
- Im SIM-Modus nur lokale Updates
- Fehlerbehandlung mit Logging
- Nutzt aiohttp asynchron



mqtt_handler.py

- `start()` → Baut WebSocket-MQTT-Verbindung auf
- `stop()` → Disconnect + Statusupdate
- `_on_message(topic, payload)` → Thread-sicheres Routing in UI
- `_stepper_increment()` → Spezialfall Drehscheibe (-60° pro Trigger)
- Subscribed alle `/PlcNode/Get/<node_id>` Topics
- Bridge zwischen MQTT-Thread und asyncio-Loop



timeline_handler.py

- subscribe() → Bindet Omniverse-Timeline-Events
- unsubscribe() → Lösen + SIM-Loop stoppen
- _on_event() → Reagiert auf PLAY / STOP
- _sim_loop() → 20Hz Loop: Deckel halten + Presse prüfen
- _reset_all_to_zero() → Setzt alle Werte/Anzeigen zurück
- Hält Greifer-Verhalten konsistent mit Timeline



routines.py

- `start_gesamtprozess()` → Startet Hauptablauf (UI-Button)
- `_run_gesamtprozess()` → 11 Phasen Choreografie
- `_run_ba_start()` → Schwenkarm-Sequenz (Pickup → Place)
- `_step(description, delay)` → Loggt + wartet
- `_set(node_id, value)` → Setzt Node (Sonderfall Sauggreifer)
- `_impulse(node_id)` → Triggert STEP-Impuls
- Race-Schutz: `_gesamt_running`, `_ba_running`



ui_builder.py

- `build(callbacks...)` → Erzeugt Hauptfenster
- `_build_header()` → Titel + Statusanzeige
- `_build_toolbar()` → Refresh / Restart / SIM↔LIVE
- `_build_column_headers()` → Spaltenüberschriften
- `_build_gesamt_button()` → Großer Prozess-Start-Button
- `_build_log_header()` → Log-Bereich mit Clear-Button
- Liefert Referenzen für externen Logik-Controller



extension.py

- `on_startup(ext_id)` → Initialisiert alle Komponenten
- `on_shutdown()` → Sauberes Aufräumen, Tasks killen
- `_refresh_nodes()` → Lädt JSON neu
- `_restart_extension()` → Extension neu starten
- `_toggle_sim_mode()` → Wechsel SIM ↔ LIVE
- `_on_control_clicked(node_id)` → Routet Toggle/Step/Velocity
- `execute_step_impulse(node_id, node)` → Step-Impuls ausführen
- `_execute_velocity_impulse(node_id, node)` → Async Velocity-Impuls
- `_set_status_text(text, color)` → Statusbar oben rechts
- Zentraler Controller, der alle Module verbindet



Nodes_db.json

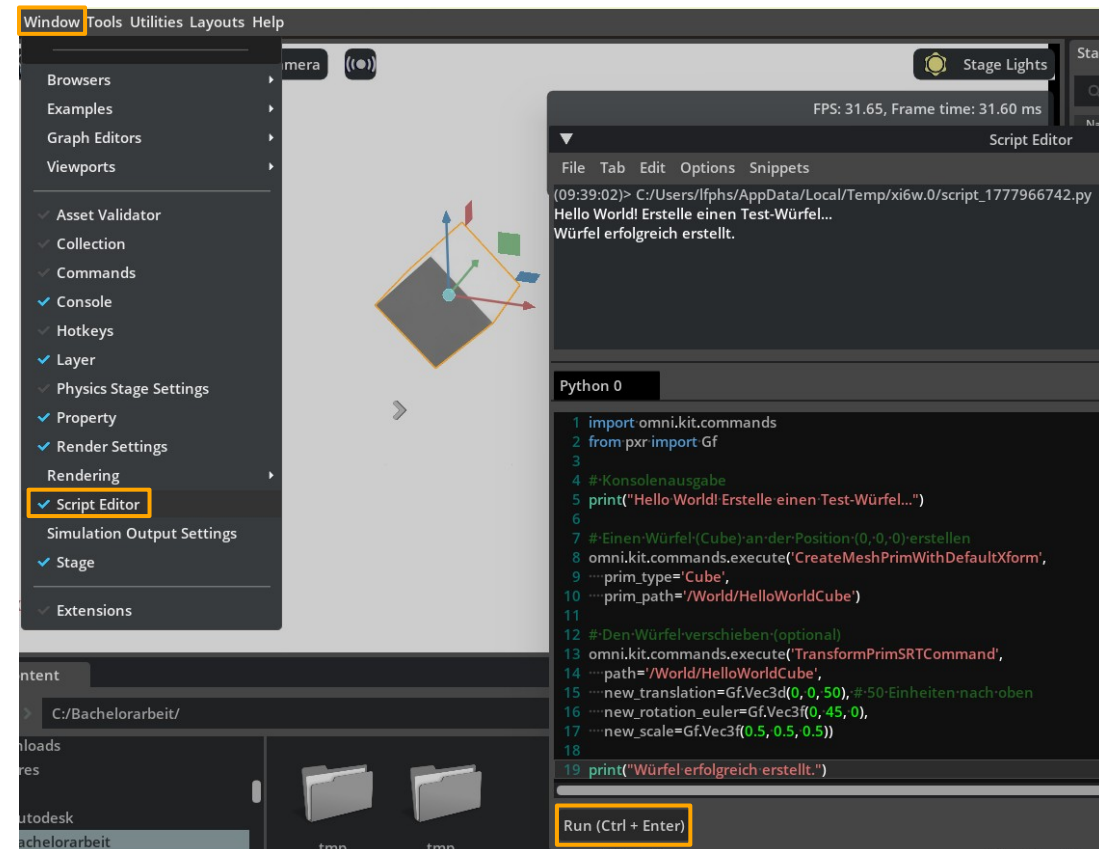
6. Neben der extension.toml liegen unter dem Pfad
omni\mazerunner\API die .py - Dateien der Extension
7. In diesem Ordner befindet sich ebenfalls die Datei „Nodes_DB.json“
Hier werden die Nodes aus DigitalTwinApp mit der Omniverse
Szene verknüpft dabei gilt folgendes:
 1. „display_name“ wird später im UI angezeigt
 2. „node_id“ muss mit DigitalTwinApp
überinstimmen
 3. „prim_path“ ist der Pfad zum Gelenk
in Omniverse
 4. „attribute“ beschreibt den Zielparameter der
manipuliert werden soll
 5. „target_value“ ist der Zielwert



```
{
  "nodes": [
    {
      "display_name": „BM_MoveFront_Set“,
      "node_id": " BM_MoveFront_Set ",
      "prim_path": "/World/Robot/Base_Joint",
      "attribute": "drive:angular:physics:targetPosition",
      "target_value": 90.0
    }
  ]
}
```

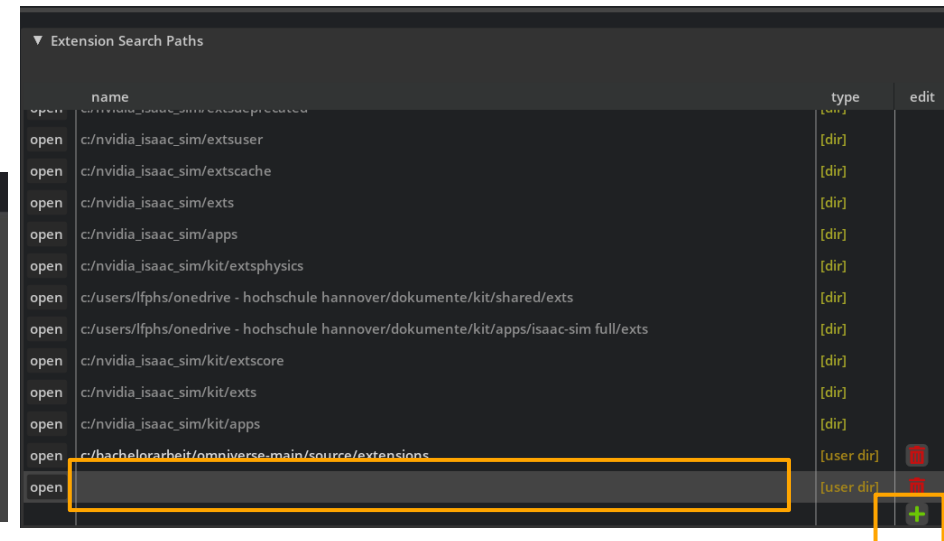
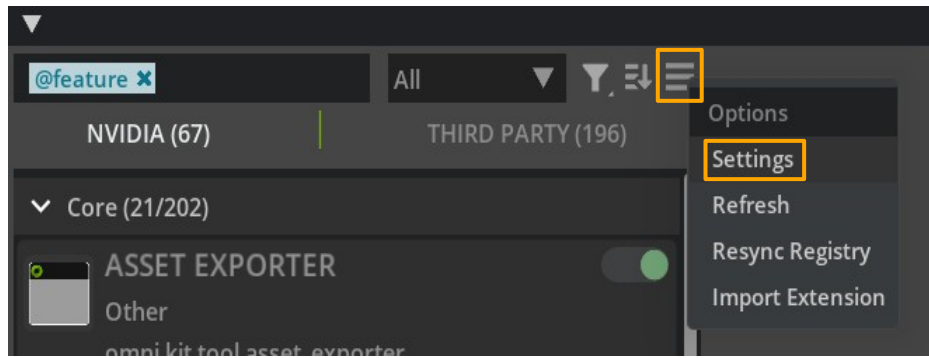
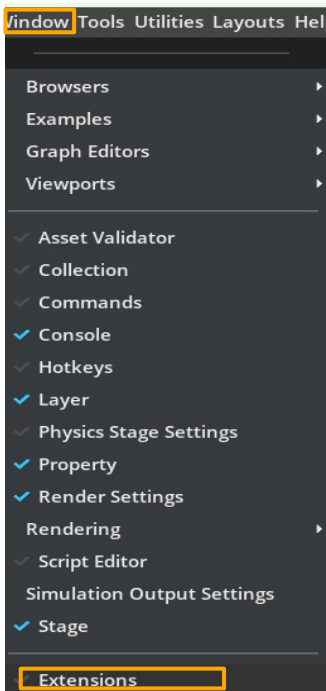
Skript Editor

1. Der Skript Editor befindet sich unter Window → Skript Editor
2. Der Code wird in das Schwarze Fenster geschrieben und mit Enter ausgeführt.
3. Zur Demonstration wurde hier ein Würfel mit dem Namen „Hello World“ durch den Skript Editor erstellt



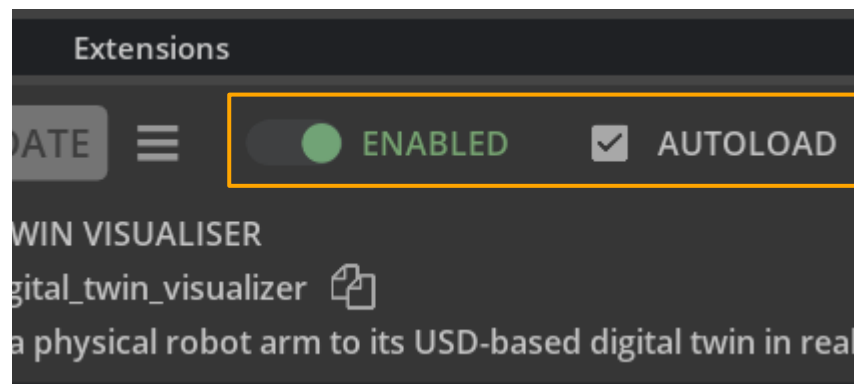
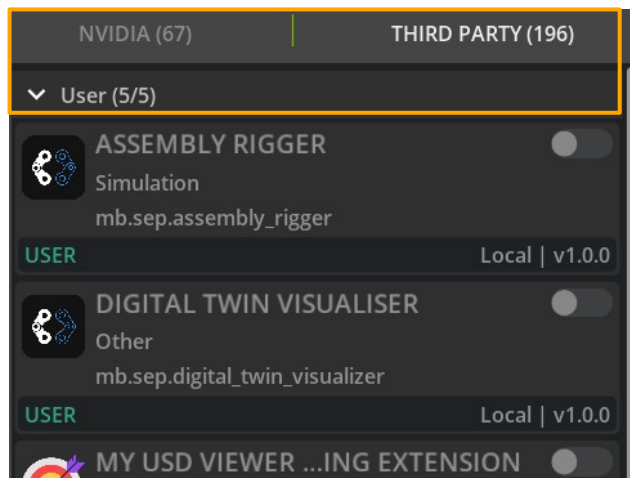
Extension aktivieren

1. Extension Datei herunterladen und in eine Ordner entpacken.
2. In Isaac Sim unter Window → Extensions auswählen.
3. Im neuen Fenster die drei Striche → Settings auswählen.
4. Hier auf das grüne Plus klicken und den Pfad zum Ordner angeben in dem die Extension liegt.



Extension aktivieren

1. Jetzt sollte im Bereich „Third Party“ unter User die neue Extension erscheinen.
2. Diese wird ausgewählt und im rechten Bereich auf „enabled“ und „autoload“ gestellt
3. Sollte es hier Probleme geben, gilt es die extension.TOML zu überprüfen.



Extension bearbeiten

1. [VS-Code](#) herunterladen
2. In VS Code durch die Datei, bis zu den Python Dateien, navigieren und diesen Ordner öffnen.
3. Die Textdatei extension.toml dient als „Visitenkarte“ der Extension für die Anzeige in Isaac Sim folgender Inhalt:

Wenn die Extension beim Aktivieren Fehlerhaft ist, liegt das meistens an dieser Datei.

```
[package]
title = "Maze Runner UI"
version = "1.0.0"

[dependencies]
"omni.kit.uiapp" = {}
"omni.ui" = {}

[[python.module]]
name = "omni.mazerunner.API"
path = "."
```



Extension Quellcode Imports

Bibliothek	Beschreibung
omni.ext	Basis-Klasse (IExt) für die Erstellung und Registrierung der Omniverse Extension.
omni.ui	Framework für das User Interface (Fenster, Buttons, Labels, ScrollingFrames).
omni.usd	Schnittstelle zur 3D-Szene, um auf USD-Prims und deren Attribute zuzugreifen.
os	Handhabung von Dateipfaden (z. B. Lokalisierung der nodes_db.json).
json	Parsen der Konfigurationsdatei und Verarbeiten der API-Antworten.
asyncio	Verwaltung asynchroner Tasks, um die Simulation nicht zu blockieren.
aiohttp	Asynchroner HTTP-Client für die Kommunikation mit der Web-API.

Extension Quellcode Aufbau

1. Die Klasse `MyExtension(omni.ext.IExt)` ist der sogenannte Main Entry Point sie erbt von der Basis Schnittstelle `IExt` und ist somit tiefgehend in Omniverse integriert
2. Standard Lifecycle Methoden müssen in jeder Extension vorkommen und beschreiben das Verhalten beim Aktivieren und Deaktivieren im Omniverse Extension Manager
3. `on_startup(ext_id)`
Initialisierung der Pfade, Aufbau der `omni.ui`-Oberfläche & Start des asynchronen API-Update-Loops
4. `On_shutdown()`
Sauberes Beenden der Hintergrund-Tasks (cancel), Schließen der UI-Fenster & Freigabe von Ressourcen.



Extension Quellcode Interne Methoden

5. Neben den notwendigen Methoden, werden einige spezifische Methoden angelegt

1. `load_nodes_from_json()`:

Liest die `nodes_db.json` ein und generiert dynamisch die UI-Elemente für jeden Knoten.

2. `auto_update_loop()`: Ein asynchroner Hintergrundprozess, der kontinuierlich Zustandsänderungen von der Web-API abfragt.

3. `on_control_clicked()`: Verarbeitet Benutzerinteraktionen (Toggle-Buttons) und stößt die Datenübertragung an die API an.

4. `update_via_web_api()`: Die eigentliche Schnittstelle, welche die Werte aus der API in die USD-Attribute (`targetPosition`) schreibt.



Funktionale Emulation mittels OPC UA Mock-Server

- Zweck: Bereitstellung einer virtuellen Gegenstelle (Server), die das Kommunikationsverhalten einer echten SPS simuliert.
- Vorteil: Entwicklung der Isaac Sim Extension unabhängig von physischer Hardware und Netzwerk-Infrastruktur.
- Test-Logik: Der Mock-Server liest die nodes_db.json aus und schaltet die hinterlegten Datenknoten (Nodes) zyklisch zwischen den Zielwerten (z. B. 0 und 1) um.
- Ziel: Validierung, ob die Extension die Werte korrekt empfängt und die digitalen Zwillinge in der Simulation entsprechend ansteuert.

```
-----
MOCK SERVER: INITIALISIERUNG
-----
[BOOL] Registriert: BA_Start      | ID: ns=4;i=4
[BOOL] Registriert: BM_MoveFront_Set | ID: ns=4;i=19
[DOUBLE/FLOAT] Registriert: DS_StartStepper_Set | ID: ns=4;i=7
[BOOL] Registriert: DM_MoveFront_Set | ID: ns=4;i=10
[BOOL] Registriert: KM_Stepper_Start | ID: ns=4;i=13
[BOOL] Registriert: Squeezer_start_set | ID: ns=4;i=16
-----
SIMULATION GESTARTET (Schrittfolge läuft...)
-----
>>> SET: BA_Start -> TRUE
<<< RESET: BA_Start -> FALSE
>>> SET: BM_MoveFront_Set -> TRUE
<<< RESET: BM_MoveFront_Set -> FALSE
>>> SET: DS_StartStepper_Set -> 90.0°
```

Property Maze Runner - OPC UA Live Monitor			
OPC UA Live-Monitor			REFRESH
Variable	IP	Node-ID	Zustand (Wert)
BA Start	192.168.60.12	ns=4;i=4	-
BM MoveFront Set	192.168.60.11	ns=4;i=19	FALSE
DS StartStepper Set	192.168.60.11	ns=4;i=7	0.00
DM MoveFront Set	192.168.60.11	ns=4;i=10	FALSE
KM Stepper Start	192.168.60.11	ns=4;i=13	TRUE
Squeezer start set	192.168.60.12	ns=4;i=16	FALSE
AA Start	192.168.60.12	ns=4;i=4	FALSE



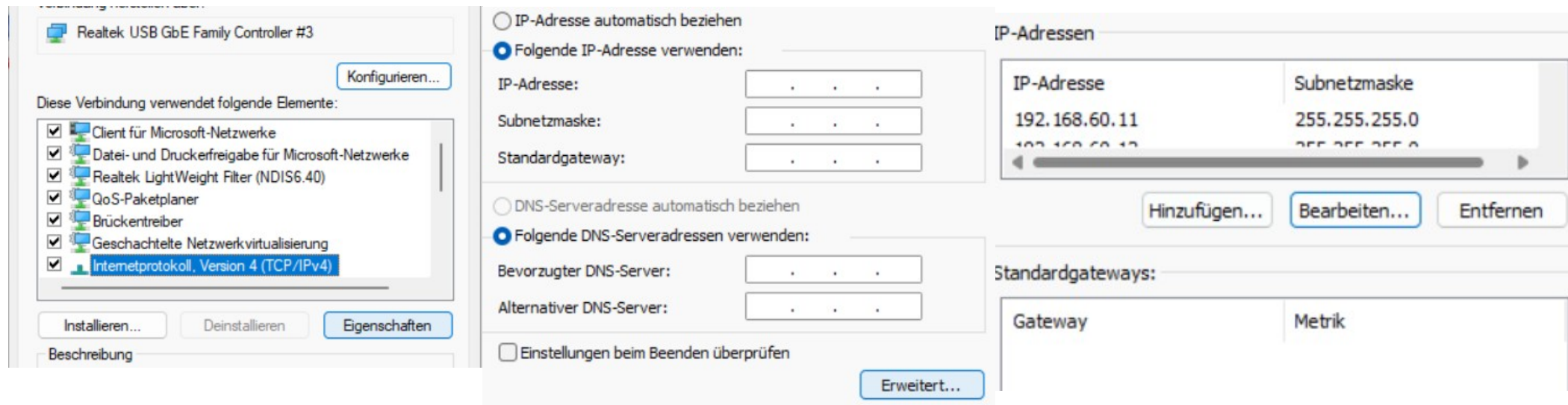
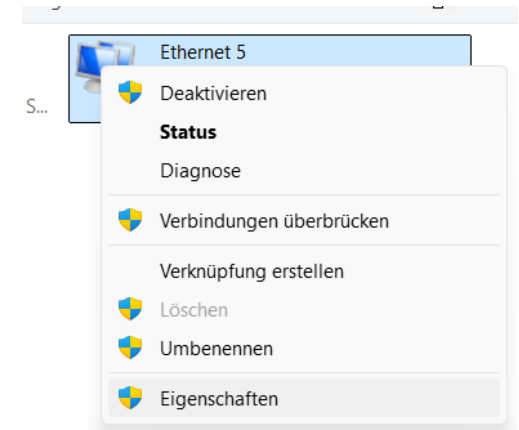
Implementierung des OPC UA Mock-Server

- IP-Adress-Zuweisung: Dem PC werden manuell die IP-Adressen der zukünftigen Hardware-Steuerungen zugewiesen (z. B. 192.168.60.11 und .12).
- Lokales Routing: Durch diese Zusatz-IPs "glaubt" das Betriebssystem, selbst die SPS-Gegenstelle zu sein.
- Kommunikationsfluss: Isaac Sim sendet Anfragen an die IP .11 oder .12. Der lokal laufende Mock-Server fängt diese Anfragen ab und antwortet mit den Test-Werten aus der Datenbank.
- Ergebnis: Die gesamte Kommunikationskette wird unter realen Netzwerkbedingungen (IP-basiert) getestet, ohne den Code später ändern zu müssen.



Leitfaden zu Inbetriebnahme des OPC UA Mock-Servers

1. Dazu Win-Taste → „Netzwerkverbindungen Anzeigen“
→ Rechtsklick auf Aktive Verbindung → „Eigenschaften“
2. „Internetprotokoll, Version 4“ → „Eigenschaften“
3. „Erweitert“ auswählen und die IP-Adressen der SPS-Anlagen einfügen
4. Eine Weitere IP-Adresse für den PC hinzufügen 192.168.60.13
5. Standardgateway freilassen



Transition zur realen Hardware

- IP-Adress-Bereinigung: Die zusätzlichen IP-Adressen (.11, .12) werden am PC in den Netzwerkeinstellungen gelöscht, um IP-Konflikte mit der echten Hardware zu vermeiden.
- Netzwerk-Kopplung: Physikalische Verbindung des PCs mit dem Automatisierungs-Netzwerk (via GL.iNet Router via Wifi).
Der PC behält dabei eine eigene, eindeutige IP (z. B. .13).
- Hardware-Aktivierung: Start des OPC UA Servers auf der realen SPS.
- Funktionstest: Da die `node_id` und die IP-Adressen in der `nodes_db.json` bereits identisch mit der Hardware konfiguriert wurden, ist keine Änderung am Programmcode der Extension erforderlich.



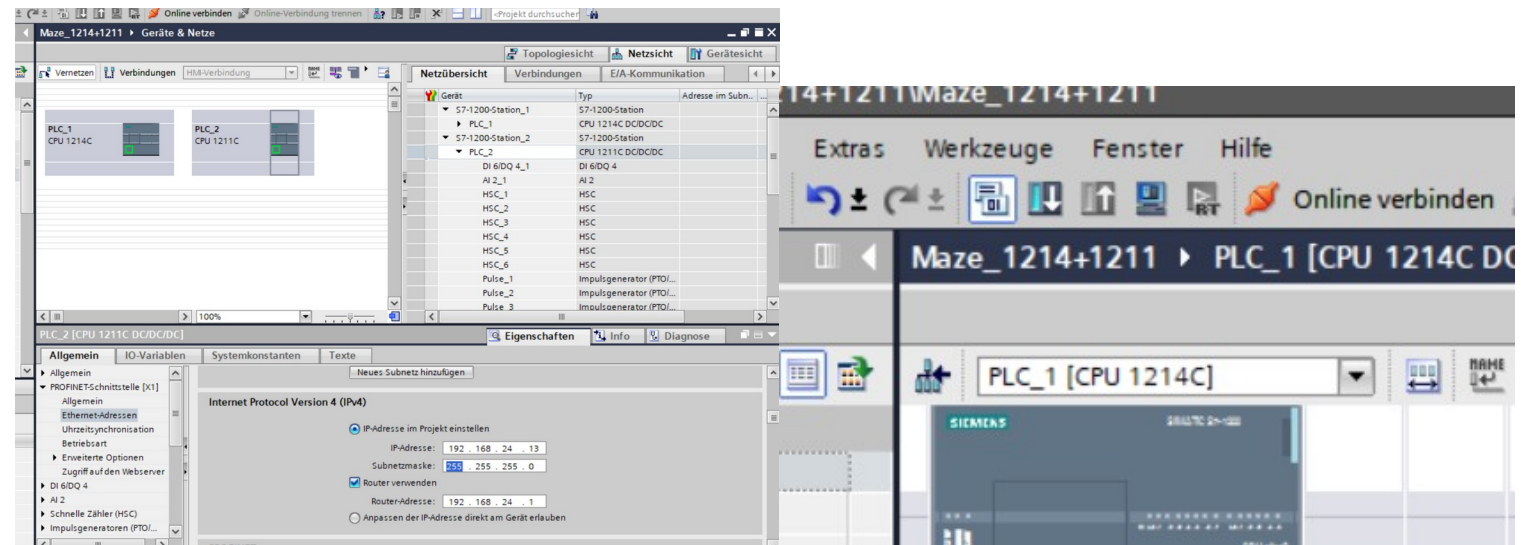
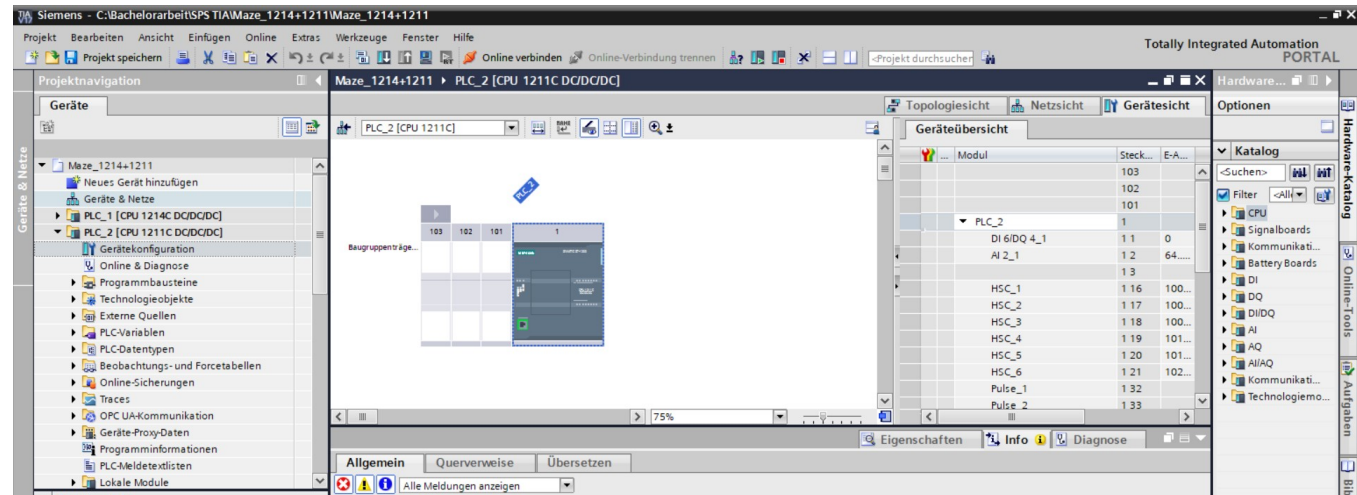
Installation

1. [TIA Portal Update 9](#) herunterladen
2. Falls [SIMATIC S7 Distributed Safety](#) beim Öffnen von TIA Portal als fehlend angezeigt wird, muss der Zugriff für den Download bei Siemens angefragt werden.

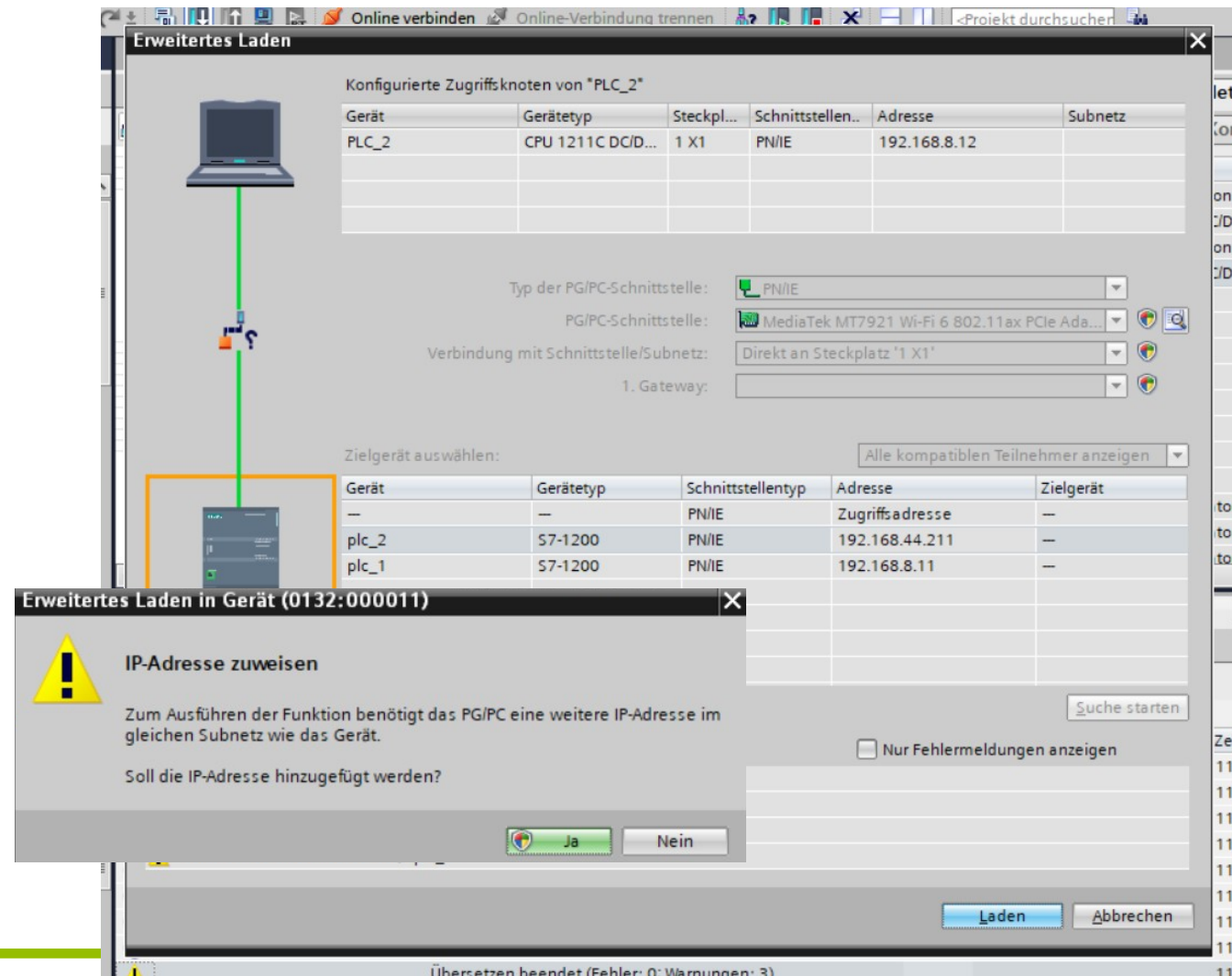


IP-Adresse der SPS Ändern

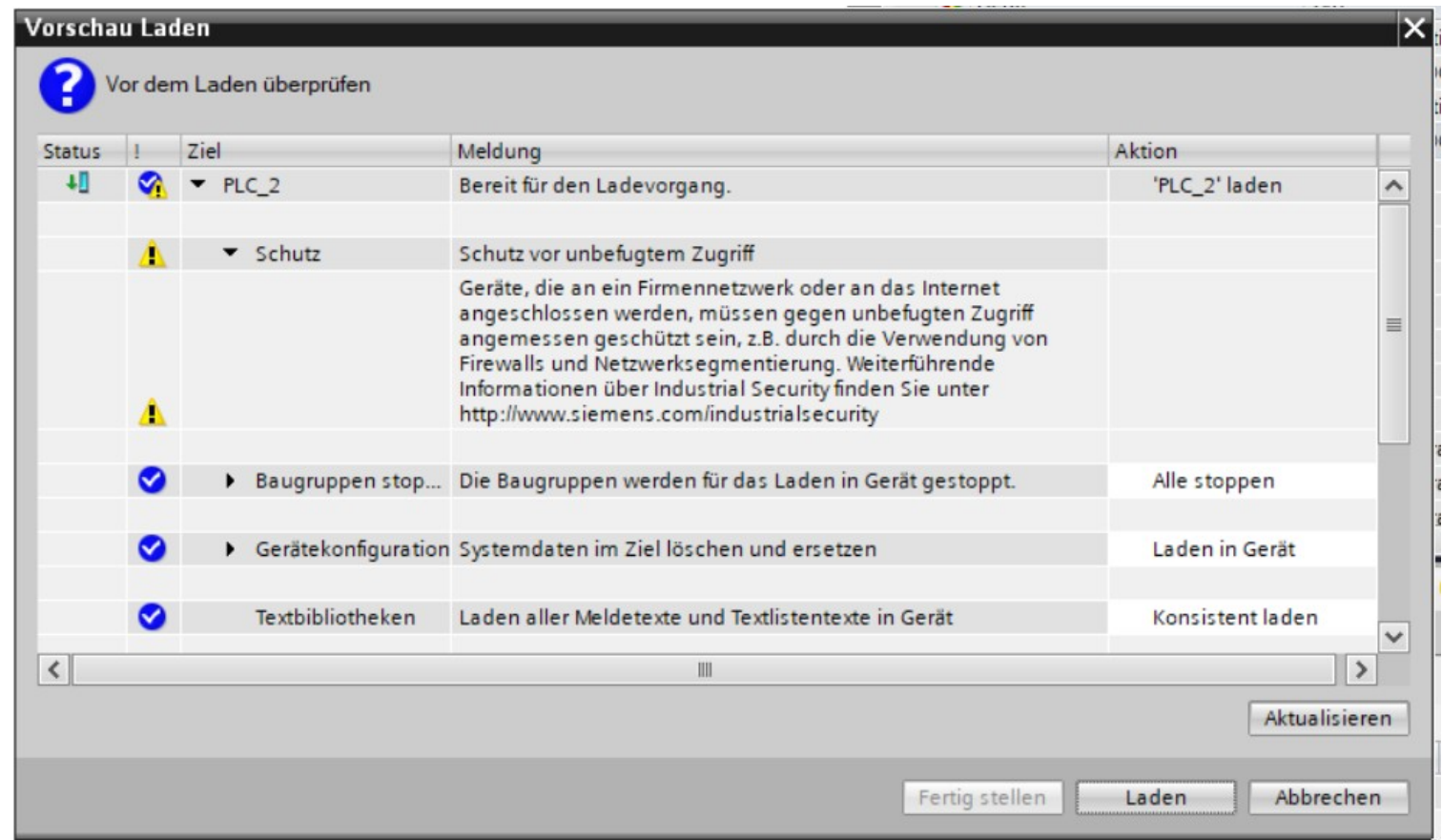
1. Online Verbindung Trennen



Verbindung zur SPS herstellen



Konfiguration Laden



MQTTL für SPS nachrüsten

[https://support.industry.siemens.com/cs/document/109780503/simatic-s7-1500-s7-1200-bibliotheken-für-kommunikation-\(lcommsuite\)?dti=0&lc=de-DE](https://support.industry.siemens.com/cs/document/109780503/simatic-s7-1500-s7-1200-bibliotheken-für-kommunikation-(lcommsuite)?dti=0&lc=de-DE)



Problembehandlung

1. Falls TIA
2. Falls [SIMATIC S7 Distributed Safety](#) beim Öffnen von TIA Portal als fehlend angezeigt wird muss der Zugriff angefragt werden.



Zusammenfassung

Die Maze-Runner-Extension wurde in elf klar abgegrenzte Module zerlegt, sodass jede Datei genau eine Verantwortung trägt und gut wartbar ist. Die `constants.py` bündelt sämtliche Konfigurationswerte sowie Farben, USD-Pfade für den Sauggreifer, Endpunkte und Geometrie-Offsets an einer zentralen Stelle. Die UI wird vollständig vom `UI_builder` aufgebaut, welcher die Layout-Logik liefert und Callbacks über Parameter entgegennimmt, dadurch bleibt das Hauptmodul frei von UI-Code. Die Datenhaltung übernimmt der `NodeManager`, der die JSON-Konfiguration lädt, Statuslabels pflegt und auf USD-Attribute schreibt. Spezialisierte Module kümmern sich um den Sauggreifer (`SuctionGripper`), die REST-API (`APIClient`), die Live-Verbindung über WebSocket-MQTT (`MQTTHandler`) und die Reaktion auf Omniverse-Timeline-Events (`TimelineHandler`). Der Gesamtprozess und die `BA_Start`-Sequenz liegen gekapselt in `routines.py`, was sie unabhängig vom Hauptmodul testbar und erweiterbar macht. Der zentrale Controller ist `extension.py`, er instanziiert die Komponenten, verbindet die UI-Callbacks und vermittelt Klicks an die jeweils zuständigen Module. Das Logging läuft thread-sicher über die `Logger-Klasse`, die sowohl aus dem `asyncio-Loop` als auch aus `MQTT-Threads` sicher angesprochen werden kann. Dadurch entsteht eine klare Trennung zwischen Darstellung, Datenmodell, Simulationslogik, externer Kommunikation und Automatisierung. Ein gut wartbares, erweiterbares Gesamtkonzept wobei noch keine zuverlässige MQTT Kommunikation dokumentiert werden kann, hier steht noch aus ob die SPS entsprechend konfiguriert werden muss, allerdings kommen vereinzelte, erwartete Nachrichten, stark verzögert in Isaac Sim an.



Quellen

1. Labor Mechatronik (Trainingslevel Konstruktion und Inbetriebnahme einer Maze Runner Produktionslinie unter Einsatz interoperabler Edge- und Cloud-Technologien) SS2024 (13.05.2026)
2. [Isaac Sim API Spezifikation](#) (12.04.2026)
3. [Isaac Sim Core](#) (01.05.2026)
4. [Isaac Sim Python API](#) (06.05.2026)
5. [Eclipse Paho MQTT Python Client](#) (13.05.2026)
6. [API – REST Requests](#) (13.05.2026)
7. [WebSocket API](#) (09.04.2026)
8. [OPC-UA Server Konfiguration](#) (18.02.2026)



Tabelle 4: Sieben-Schritte-Modell zur Reproduktion eines Digitalen Zwillings

Nr.	Schritt	Artefakt
1	Anforderungs- und Schnittstellenanalyse	Anforderungsliste, Schnittstellenmatrix
2	CAD-Aufbereitung und USD-Export	.usd-Stage
3	Definition kinematischer Strukturen	Articulation-Konfiguration
4	Auswahl Middleware	Architekturdiagramm
5	Implementierung der Datenkopplung	Python-Extension, Konfigurationsdateien
6	Validierung	Messprotokolle, Latenzwerte
7	Übergabe und Versionierung	Git-Repository, Dokumentation

E. Schnittstellenmatrix Maze Runner (Auszug)

Tabelle 5: Auszug aus der OPC-UA-Schnittstellenmatrix des Maze Runner

Variable	Datentyp	Richtung	Beschreibung
TableAngle	Float	R	Aktueller Drehwinkel des Rundtaktisches in Grad
TableSpeed	Float	R/W	Soll-/Ist-Drehzahl in U/min
GripperState	Bool	R/W	Pneumatischer Greifer geöffnet/geschlossen
SensorStation1	Bool	R	Induktiver Näherungsschalter Station 1
EmergencyStop	Bool	R	Status Not-Halt-Kreis